

**Join the lecture online on your
dashboard.**

RNNs Continued

LSTMs and GRUs

Were RNNs All We Needed?

Leo Feng
Mila – Université de Montréal & Borealis AI
leo.feng@mila.quebec

Frederick Tung
Borealis AI
frederick.tung@borealisai.com

Mohamed Osama Ahmed
Borealis AI
mohamed.o.ahmed@borealisai.com

Yoshua Bengio
Mila – Université de Montréal
yoshua.bengio@mila.quebec

Hossein Hajimirsadeghi
Borealis AI
hossein.hajimirsadeghi@borealisai.com

Abstract

The introduction of Transformers in 2017 reshaped the landscape of deep learning. Originally proposed for sequence modelling, Transformers have since achieved widespread success across various domains. However, the scalability limitations of Transformers—particularly with respect to sequence length—have sparked renewed interest in novel recurrent models that are parallelizable during training, offer comparable performance, and scale more effectively. In this work, we revisit sequence modelling from a historical perspective, focusing on Recurrent Neural Networks (RNNs), which dominated the field for two decades before the rise of Transformers. Specifically, we examine LSTMs (1997) and GRUs (2014). We demonstrate that by simplifying these models, we can derive minimal versions (minLSTMs and minGRUs) that (1) use fewer parameters than their traditional counterparts, (2) are fully parallelizable during training, and (3) achieve surprisingly competitive performance on a range of tasks, rivalling recent models including Transformers.

4		Long Short-Term Memory (LSTM), Clearly Explained StatQuest with Josh Starmer
5		What is LSTM (Long Short Term Memory)? IBM Technology
6		LSTM Long Short Term Memory Part 1 The What? CampusX CampusX
7		Lecture 10 Recurrent Neural Networks Stanford University School of Engin...
		LSTM: The Comeback Story? [Prof. Sepp Hochreiter] Machine Learning Street Talk
9		Illustrated Guide to LSTM's and GRU's: A step by step explanation The AI Hacker
10		Tutorial 34- LSTM Recurrent Neural Network In Depth Intuition Krish Naik
11		xLSTM: Extended Long Short-Term Memory Yannick Kilcher
12		LSTM is dead. Long Live Transformers! Seattle Applied Deep Learning
13		CS231n Winter 2016: Lecture 10: Recurrent Neural Networks,... Andrej Karpathy

Customers Review 2,491

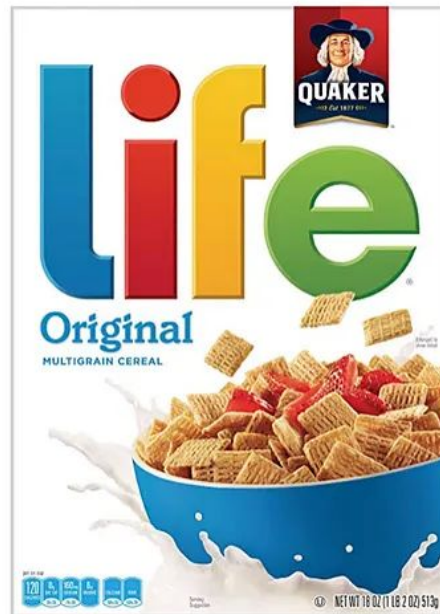


Thanos

September 2018

Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal
\$3.99

Customers Review 2,491

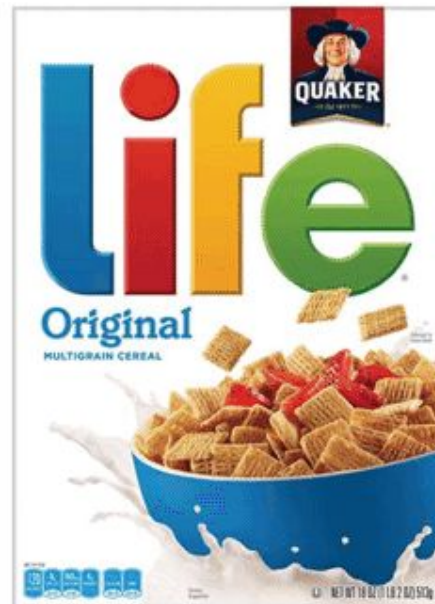


Thanos

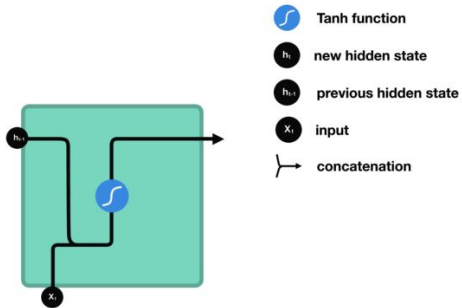
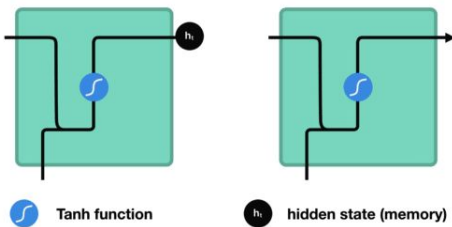
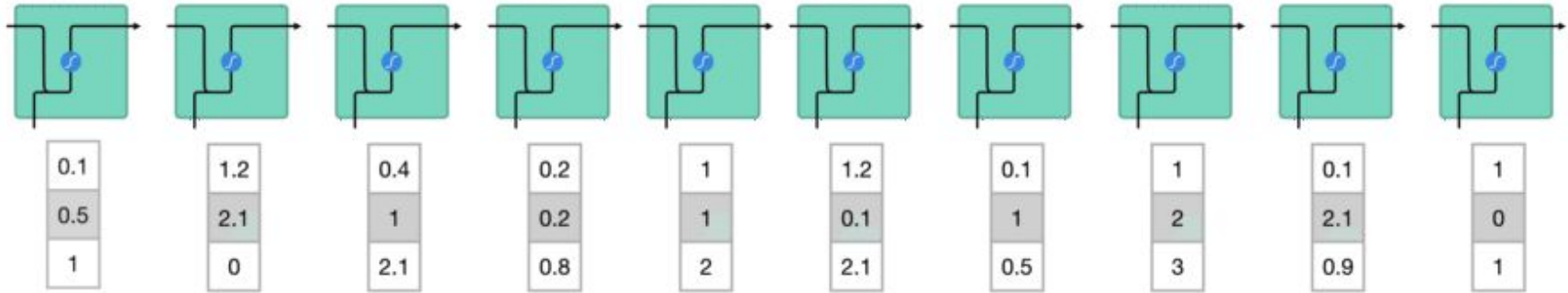
September 2018

Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal
\$3.99

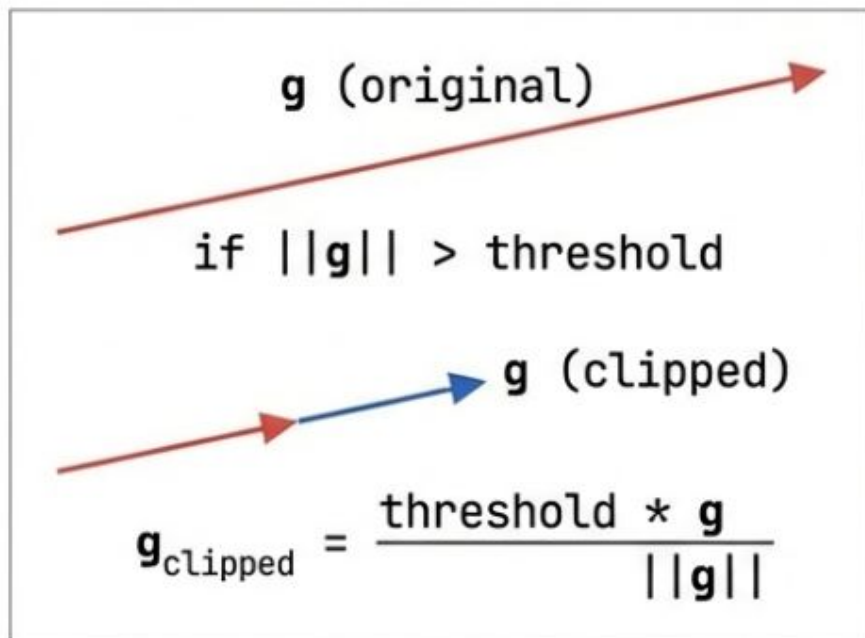


Solution of Exploding and Vanishing Gradients

Exploding gradient

Gradient clipping

- Used to avoid exploding gradient problem.
- Preserves the direction.
- Reduces the magnitude.



Like taking small, careful steps
when walking down a steep hill.

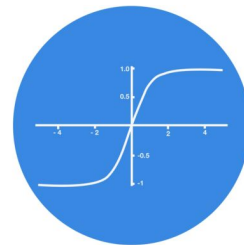
Vanishing gradient - Gated RNNs (LSTM)

- To be discussed today

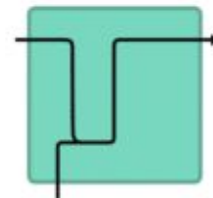
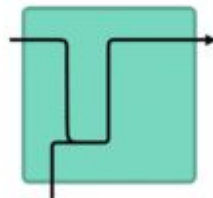
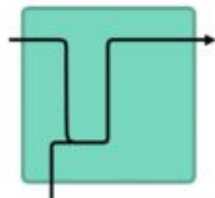
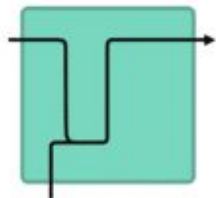
Tanh activation

The tanh activation is used to help regulate the values flowing through the network. The tanh function squishes values to always be between -1 and 1.

5
0.1
-0.5

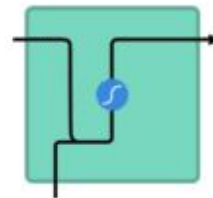
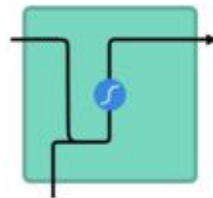
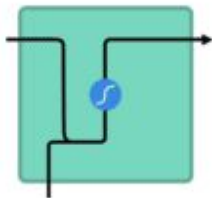
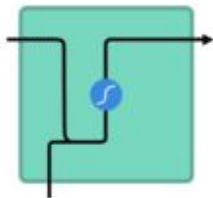


5
0.1
-0.5



vector transformations without tanh

5
0.1
-0.5



vector transformations with tanh

Problems with RNN

- Encoding bottleneck h_t
- “Imagine you are told to summarize a 3-hr movie in 1 sentence.”
- h_t is essentially a lossy compression of history.

Problems with RNN

- **Slow to train and no parallelization**
- Computations are done sequentially with time-steps.
- Difficult to parallelize.
- Unlike transformers, where all tokens can be processed simultaneously.
- Too slow to train due to Backpropagation through time.

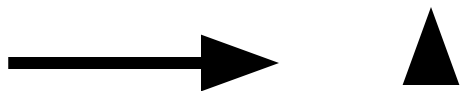
Long term dependency issue

- Even if the gradients do not vanish, RNNs are naturally biased towards recent inputs.
- Model tends to forget inputs very far in the past.
- Becomes less effective for long sequences.

Bidirectional RNN

He went to the **bank** to withdraw money.

He went to the **bank** to sit by the water.



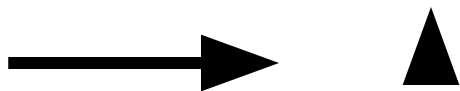
Can you find which bank relates to money or river?



Bidirectional RNN

He went to the **bank** to withdraw money.

He went to the **bank** to sit by the water.



Answer is NO.

You need future words to find the context.

Bidirectional RNN

A standard RNN will only guess the next word here.

Instead of :

What do I know from the past?

We ask:

What do I understand from the entire sequence?

 **This dramatically improves contextual understanding.**

The Big Picture

Standard RNNs assume  **The future is unavailable.**

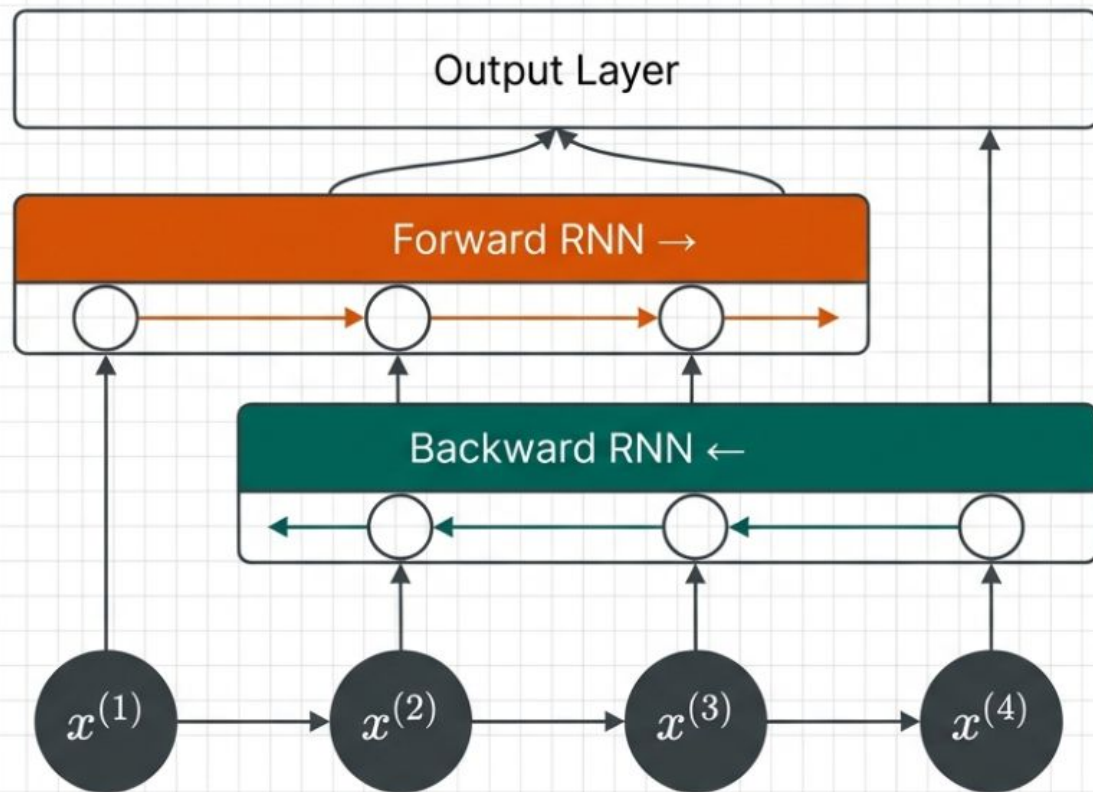
Bidirectional RNNs assume  **The full sequence is available.**

Bidirectional RNN

Bidirectional RNNs improved performance in:

- Speech recognition
- Machine translation
- Parts of speech tagging
- Named entity recognition
- Handwriting recognition

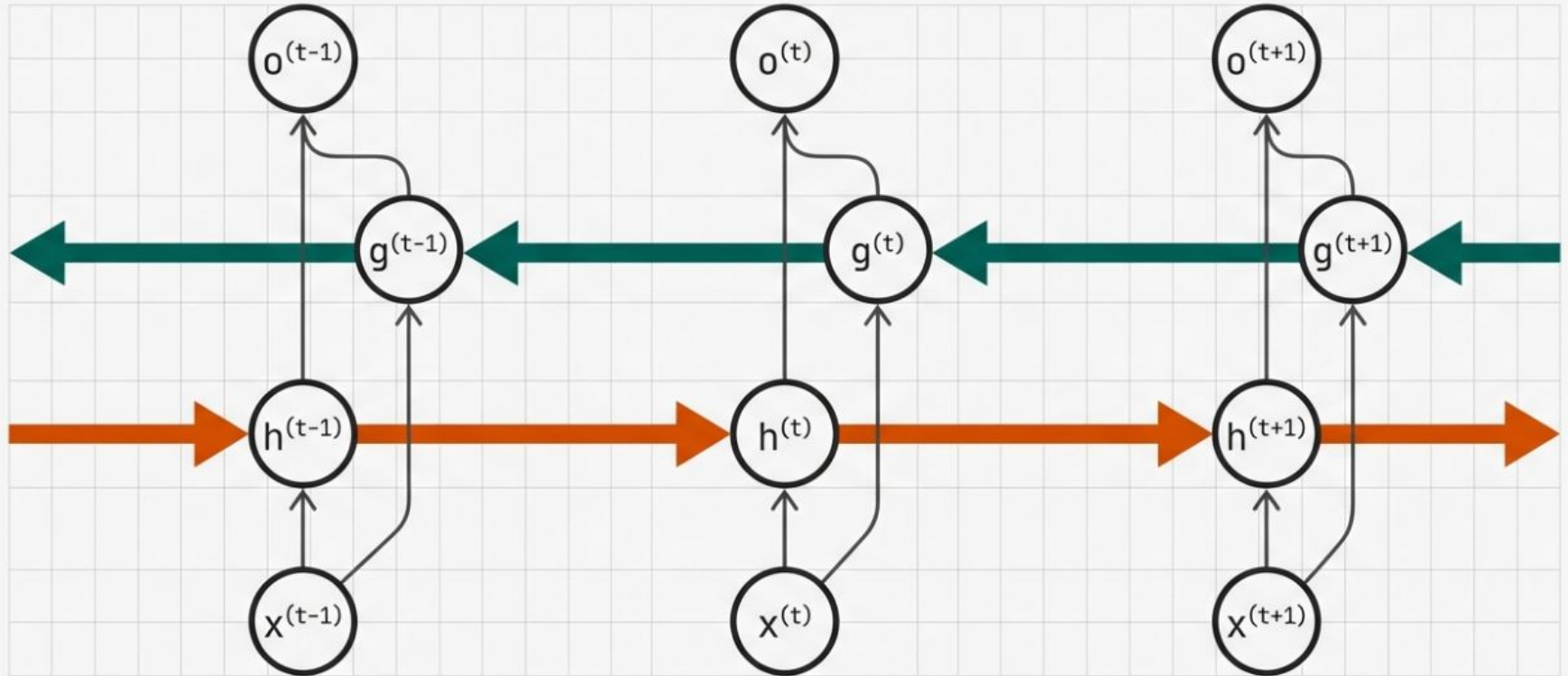
Bidirectional RNN



Architecture:

- Two separate RNNs operating on the same input sequence.
- Forward Net: Reads $x^{(1)} \rightarrow x^{(\tau)}$
- Backward Net: Reads $x^{(\tau)} \rightarrow x^{(1)}$
- They do not interact until the output layer.

The Unfolded Computational Graph



Bidirectional RNN

Intuition:

1. Read the entire sequence **forward**.
2. Read the entire sequence **backward**.
3. Now decide what each word means.

Consequences

This is why BiRNNs cannot be used for streaming or real-time tasks.

1. We must wait until the entire sequence is available.
2. We cannot produce early outputs.

Forward propagation equation

$$\vec{h}_t = \phi(W_{xh}^{(f)} x_t + W_{hh}^{(f)} \vec{h}_{t-1} + b^{(f)})$$

$$\overleftarrow{h}_t = \phi(W_{xh}^{(b)} x_t + W_{hh}^{(b)} \overleftarrow{h}_{t+1} + b^{(b)})$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

$$y_t = \psi(W_y h_t + b_y)$$

Long-term dependency issue

Predict the next word:

I grew up in France. I speak
fluent _____.

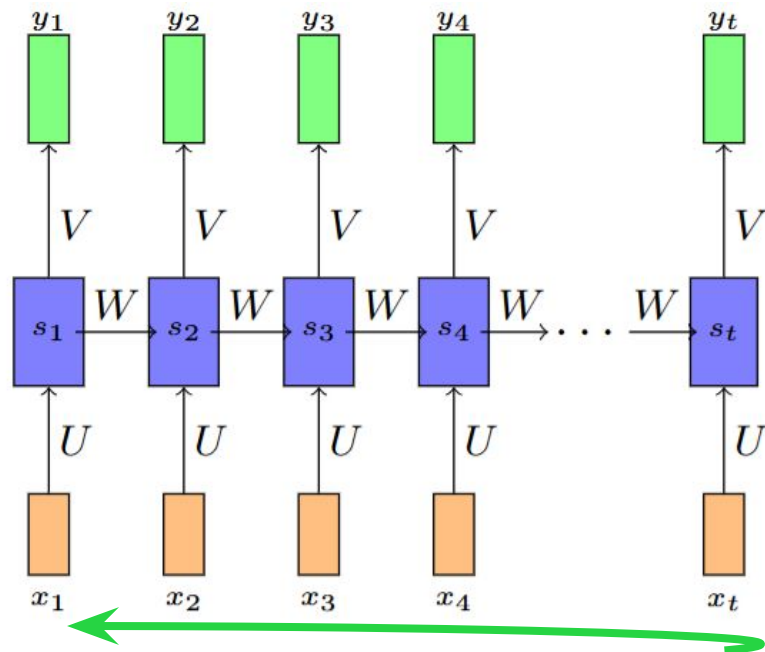
To answer correctly (**French**), the model must:

- Store the information “France”
- Keep it across many unrelated words.

Theoretically RNNs do this, practically they don't.

Long-term dependency issue

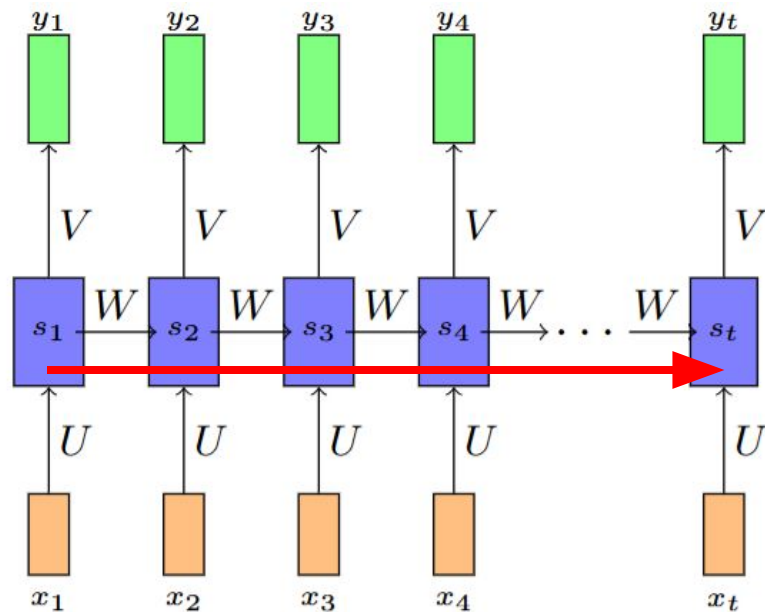
- At timestep t , s_t or h_t holds information about past.
- s_t gets transformed or morphed by each new input.
- After a very long sequence s_t can't hold information from distant past.



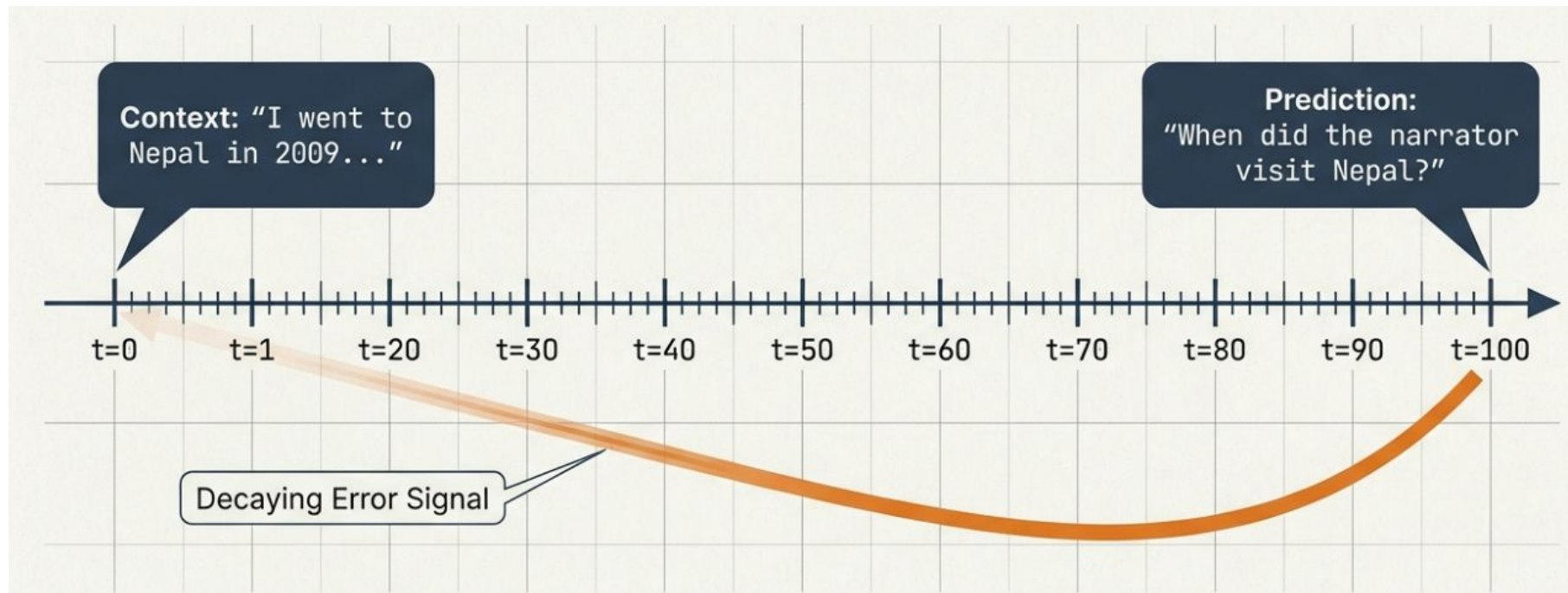
Can't remember

Long-term dependency issue

- Only one path through which the memory (context) gets updated and transferred.
- No concept of Long-Term Context and Short-Term Context.



Long-term dependency issue

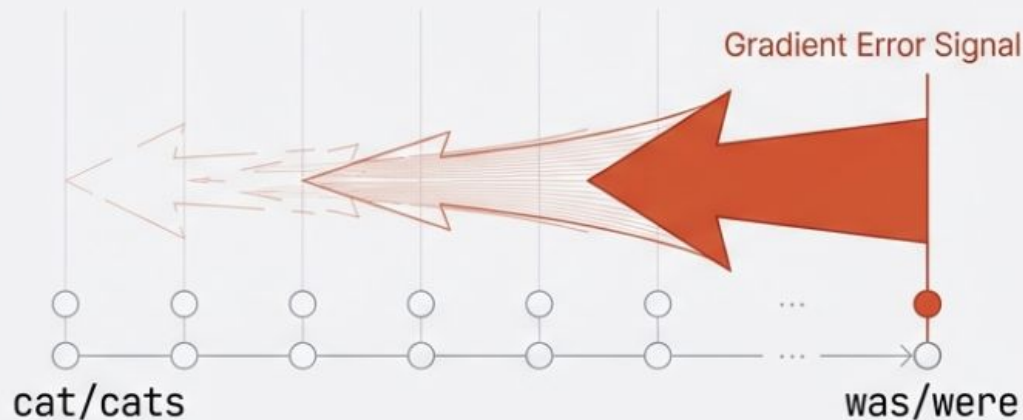


The difficulty is not about storing information only but propagating the error signal back to the point where the information was first encountered

Long-term dependency issue

Vanishing Gradients

The **cat**, which already ate..., **was** full.
The **cats**, which already ate..., **were** full.



Long-Term Amnesia: As the error signal backpropagates through time, repeated multiplication of small derivatives causes the gradient to shrink towards zero. The network learns "was" matches "ate", but forgets it needs to match "cat".

Whiteboard analogy

Consider a fixed size whiteboard:



- We are writing new information at periodic intervals.
- Effectively at each timestep, information is getting transformed or morphed.
- After a long time, it is difficult to track, how information at timestep $t - k$, i.e. x_{t-k} contributed to x_t .

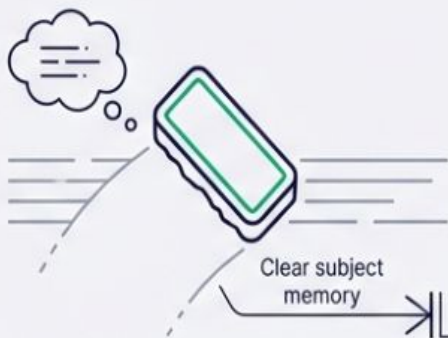
The Intuition: Selective Information Flow



Selective Write

Decide what to store.

Is this new info important (singular subject) or noise (a comma)?



Selective Forget

Decide what to discard.

The sentence ended.
Clear the subject memory.



Selective Read

Decide what to use.

For this specific prediction,
which part of memory matters?

LONG SHORT-TERM MEMORY

NEURAL COMPUTATION 9(8):1735–1780, 1997

Sepp Hochreiter
Fakultät für Informatik
Technische Universität München
80290 München, Germany
hochreit@informatik.tu-muenchen.de
<http://www7.informatik.tu-muenchen.de/~hochreit>

Jürgen Schmidhuber
IDSIA
Corso Elvezia 36
6900 Lugano, Switzerland
juergen@idsia.ch
<http://www.idsia.ch/~juergen>

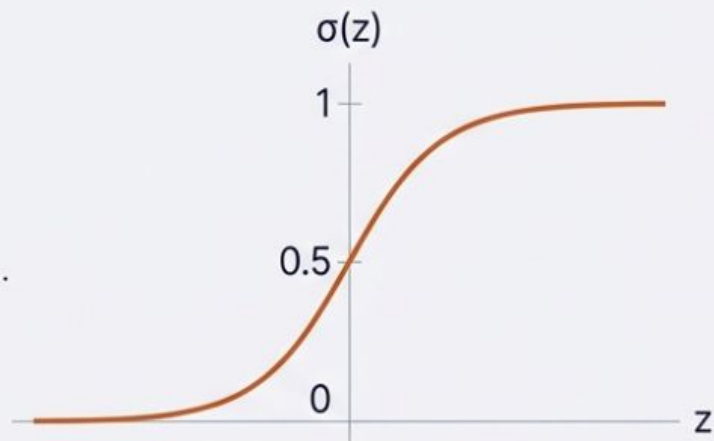
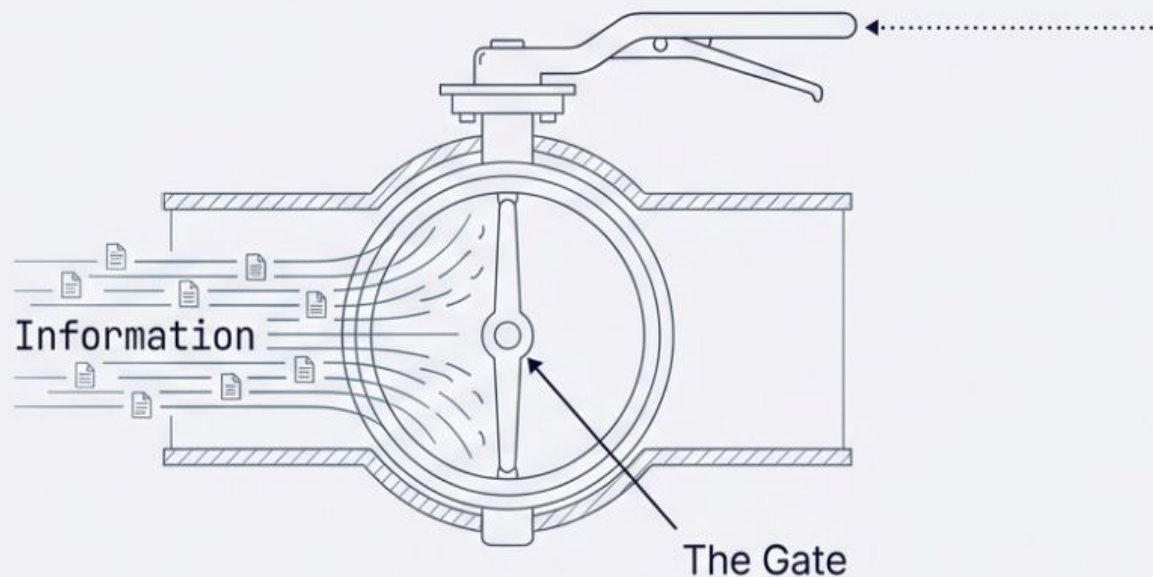
Abstract

Learning to store information over extended time intervals via recurrent backpropagation takes a very long time, mostly due to insufficient, decaying error back flow. We briefly review Hochreiter's 1991 analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called "Long Short-Term Memory" (LSTM). Truncating the gradient where this does not do harm, LSTM can learn to bridge minimal time lags in excess of 1000 discrete time steps by enforcing *constant* error flow through "constant error carousels" within special units. Multiplicative gate units learn to open and close access to the constant error flow. LSTM is local in space and time; its computational complexity per time step and weight is $O(1)$. Our experiments with artificial data involve local, distributed, real-valued, and noisy pattern representations. In comparisons with RTRL, BPTT, Recurrent Cascade-Correlation, Elman nets, and Neural Sequence Chunking, LSTM leads to many more successful runs, and learns much faster. LSTM also solves complex, artificial long time lag tasks that have never been solved by previous recurrent network algorithms.

1 INTRODUCTION

Recurrent networks can in principle use their feedback connections to store representations of

The Mechanism: The Sigmoid “Faucet”



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Value = 0



Block (Forget)

Value = 1



Pass (Remember)

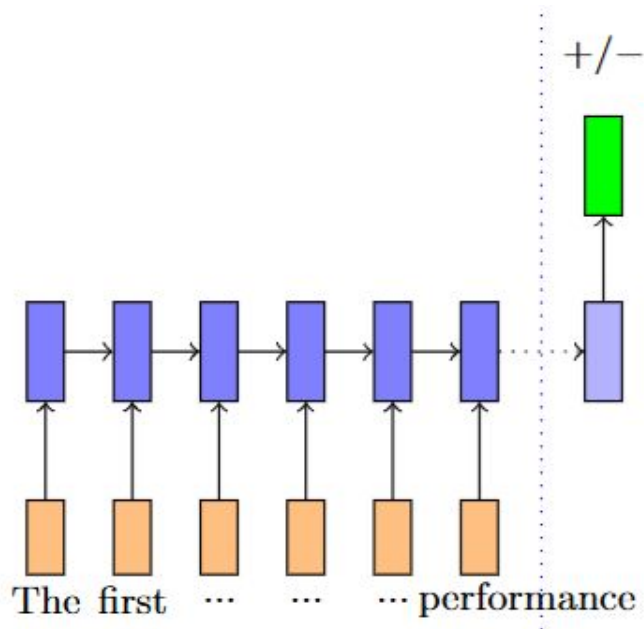
Value = 0.5



Scale

Gates learn dynamic weights (0 to 1) that change at every time step based on context.

Selective read, write, forget example



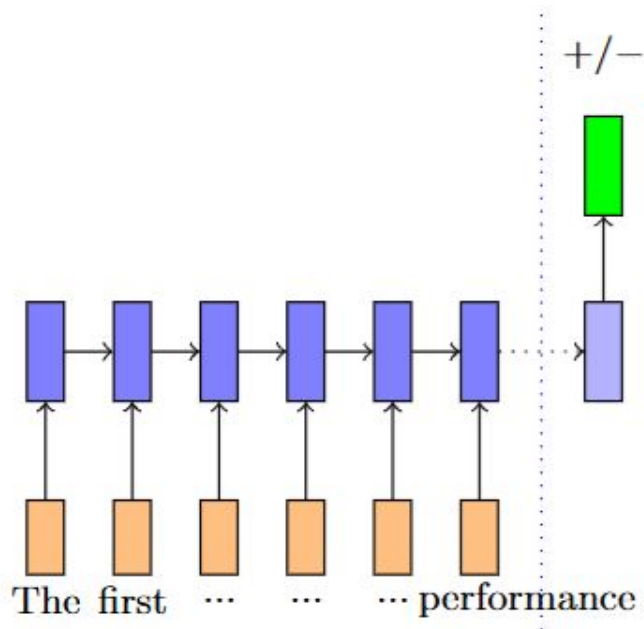
As we read the sentence from L-to-R:

We want to **forget**:

*The information added by stop words.
(a, an, the, etc.)*

Review: The first half of the movie was dry but the second half really picked up pace. The lead actor delivered an amazing performance

Selective read, write, forget example



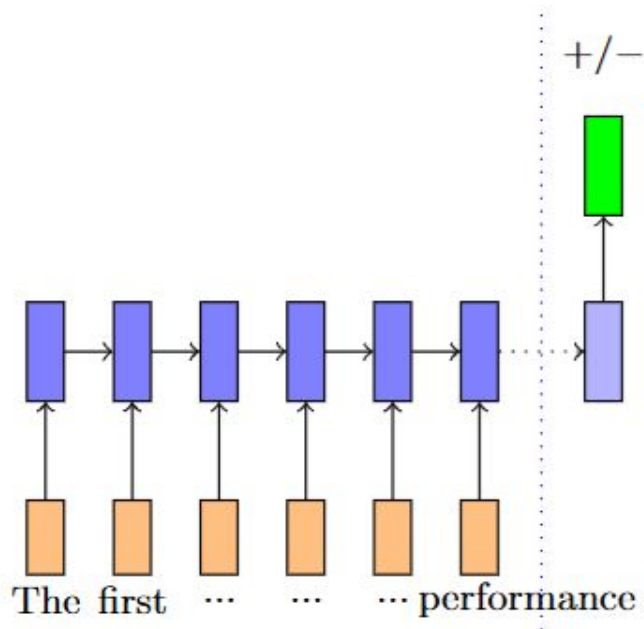
As we read the sentence from L-to-R:

We want to **selectively read**:

The information added by previous sentiment bearing words (awesome, amazing, etc.)

Review: The first half of the movie was dry but the second half really picked up pace. The lead actor delivered an amazing performance

Selective read, write, forget example

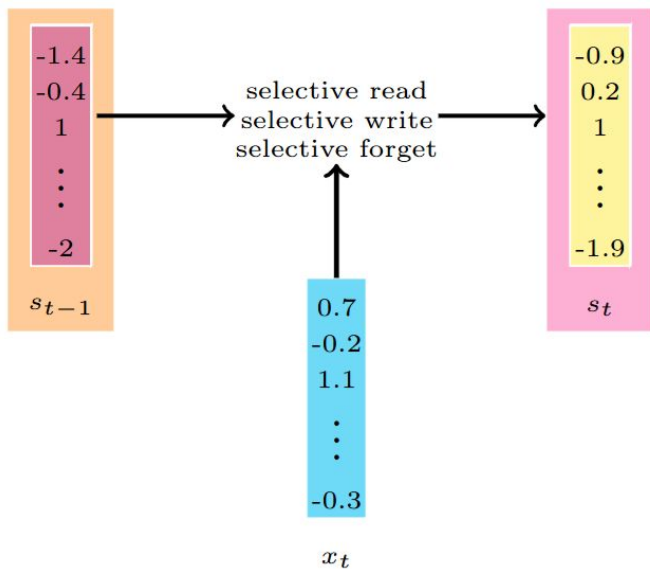


As we read the sentence from L-to-R:

We want to **selectively write**:

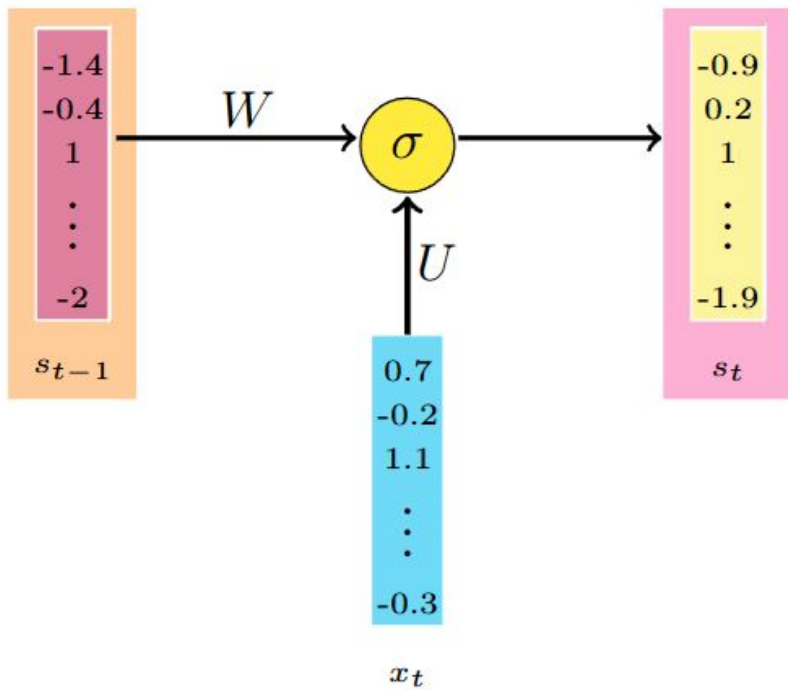
New information from the current word to the state.

Review: The first half of the movie was dry but the second half really picked up pace. The lead actor delivered an amazing performance



Problem statement

- We have computed s_{t-1} at timestep $t - 1$.
- We want to combine it with new information x_t .
- To compute a new state s_t .
- Using:
 - *Selective write*
 - *Selective read*
 - *Selective forget*
- To retain only important information.

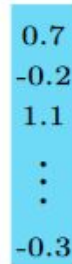
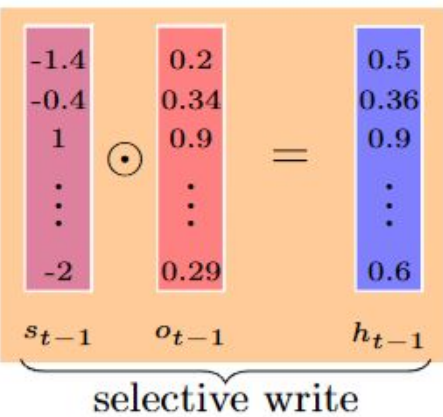


Selective write

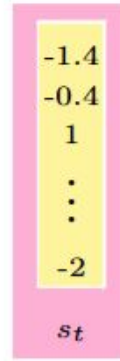
- In RNNs we have used s_{t-1} to compute s_t , as

$$s_t = \sigma(Ws_{t-1} + Ux_t) \text{ (ignoring bias)}$$

But how do we pass only a portion of s_{t-1} ?



x_t



Selective write

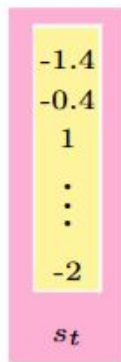
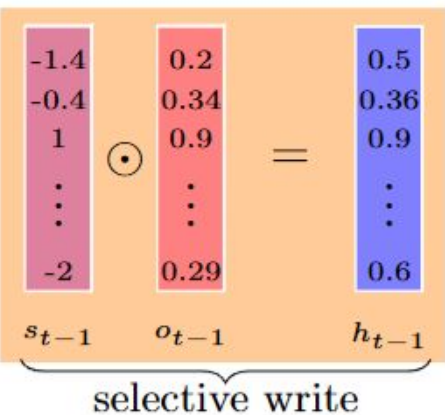
- Introduce a vector o_{t-1} .
- This is **output gate** which will control the flow of information.

But how does the network know how much information be allowed to flow?

OR

How to find o_{t-1} ?

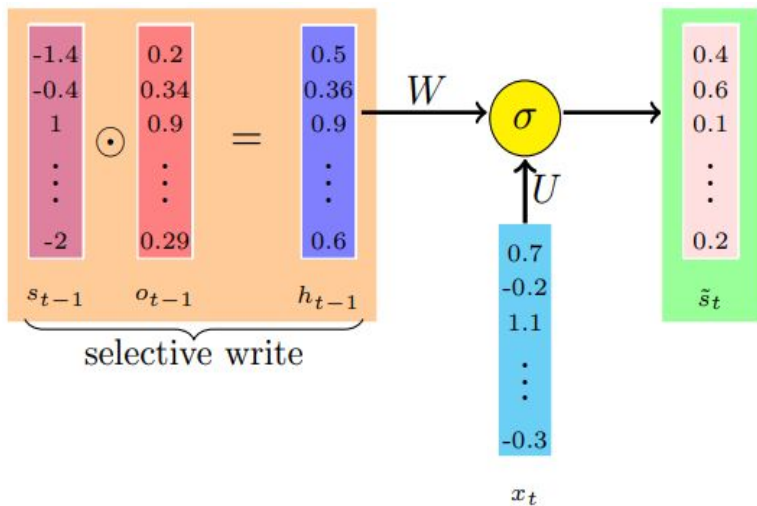




Selective write

- RNN will learn o_{t-1} by introducing learnable parameters.
- $o_{t-1} = \sigma(W_o h_{t-2} + U_o x_{t-1} + b_o)$
- $h_{t-1} = o_{t-1} \odot \sigma(s_{t-1})$
- Learnable parameters:
- W, U, V, W_o, U_o, b_o

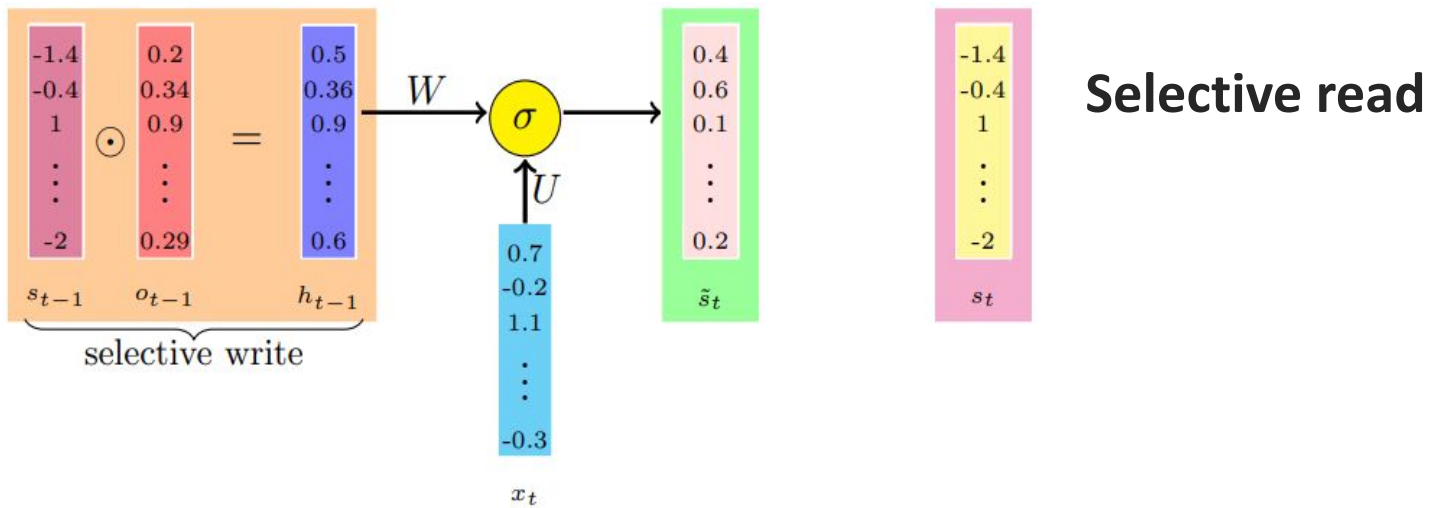
This sigmoid is just for scaling s_{t-1} .



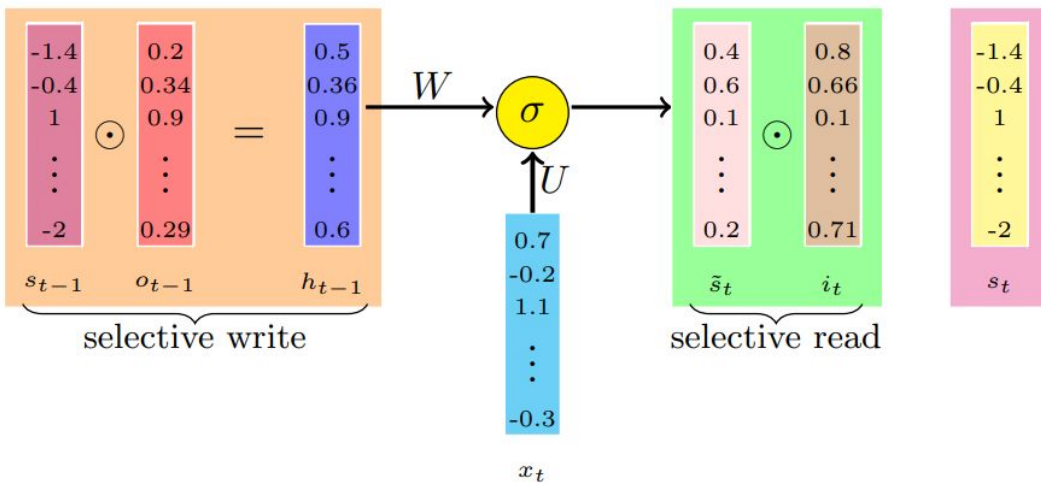
Selective read

- Use h_{t-1} to compute next state.
- Compute a new temporary state:

$$\tilde{s}_t = \sigma(Wh_{t-1} + Ux_t + b)$$
- $\tilde{s}_t$ captures information from previous state h_{t-1} and current input x_t .

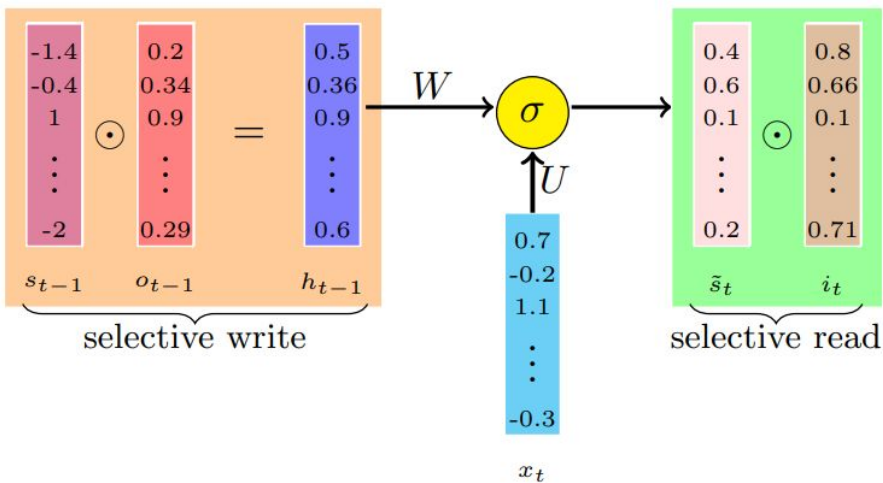


Should we use $\tilde{s}_t$ completely or selectively for computing new cell state?



Selective read

- To read selectively we'll introduce an input gate i_t .
- $i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$
- Use $i_t \odot \tilde{s}_t$ to read selective information.



So
far...

Previous state:

$$s_{t-1}$$

Output gate:

$$o_{t-1} = \sigma(W_o h_{t-2} + U_o x_{t-1} + b_o)$$

Selective write:

$$h_{t-1} = o_{t-1} \odot \sigma(s_{t-1})$$

Current (temporary) state:

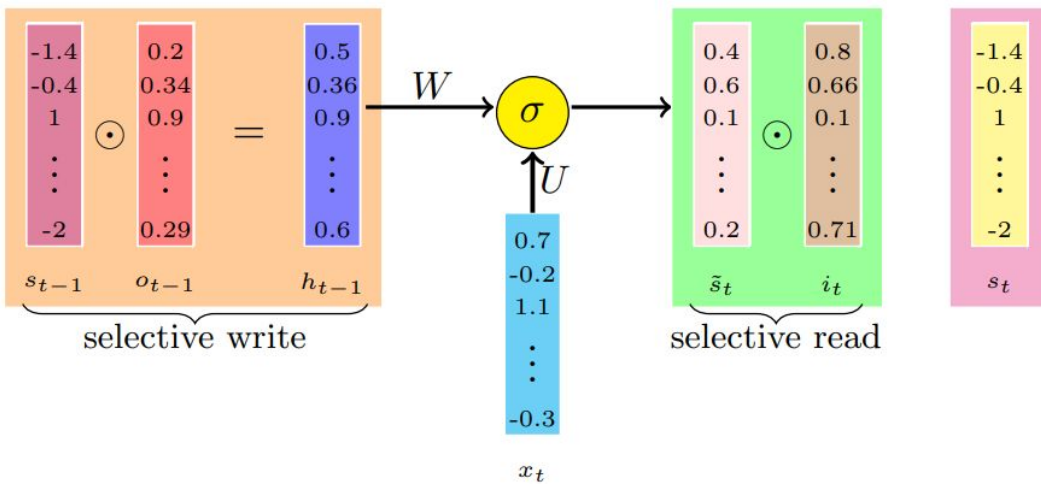
$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$$

Input gate:

$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

Selectively read:

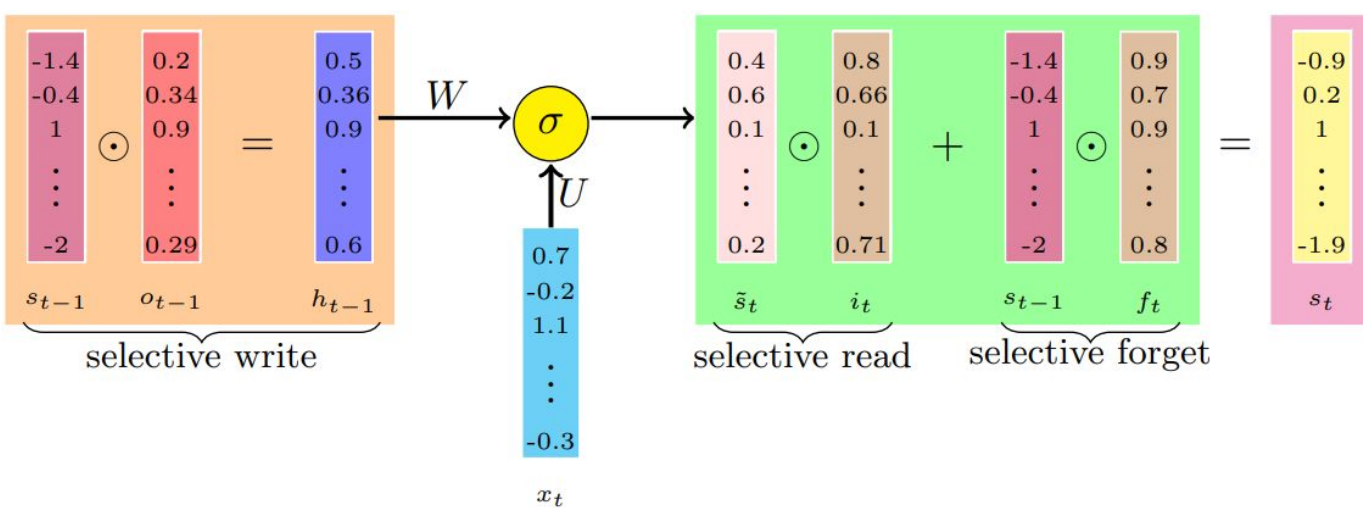
$$i_t \odot \tilde{s}_t$$



We have:

- Previous state s_{t-1} .
- Current temporary state $\tilde{s}_t$.

How to combine these to find s_t (next state)? (Selective forget)



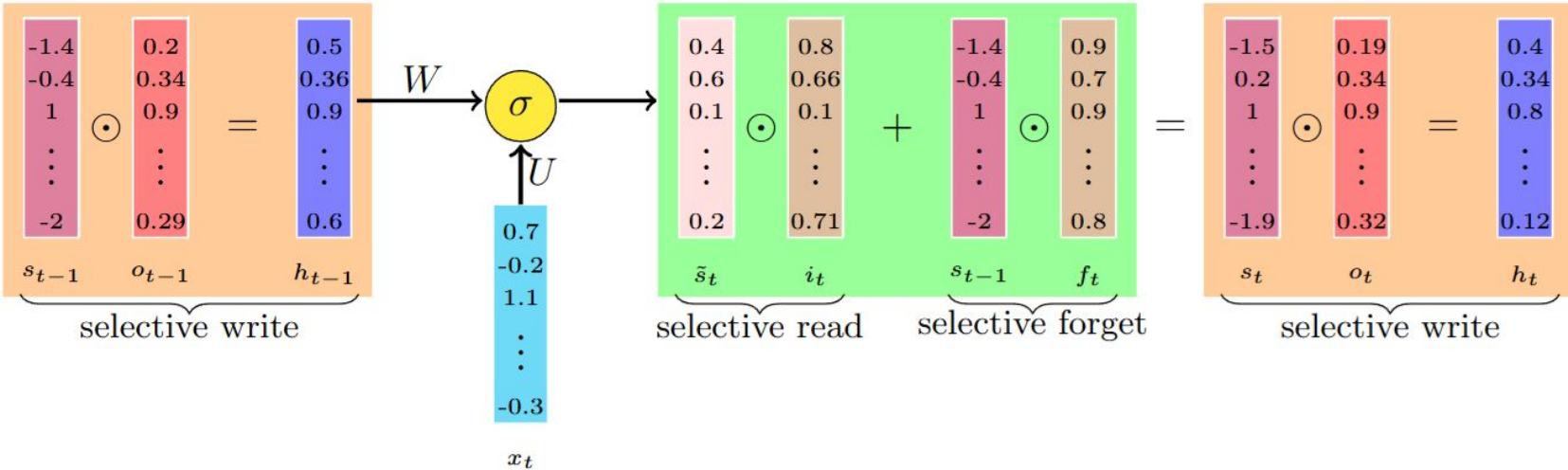
We may not need the **whole** s_{t-1} i.e.

We need to **forget** some **portion** of s_{t-1} .

Introduce the forget gate f_t :

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

$$\text{New state: } s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$$



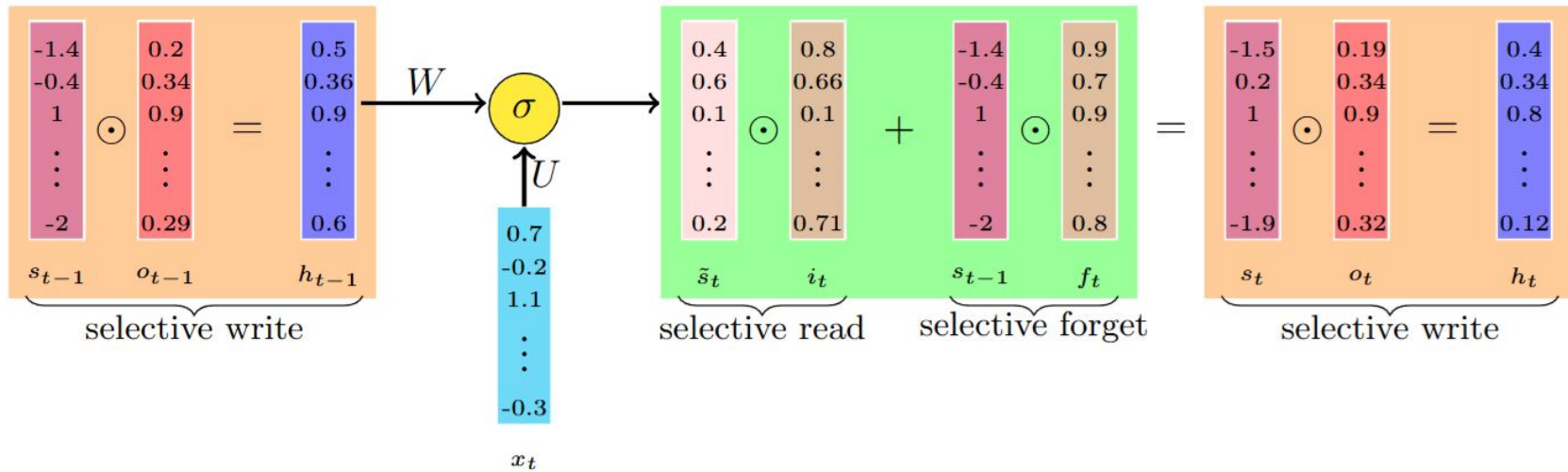
Gates:

$$\begin{aligned}
 o_t &= \sigma(W_o h_{t-1} + U_o x_t + b_o) \\
 i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i) \\
 f_t &= \sigma(W_f h_{t-1} + U_f x_t + b_f)
 \end{aligned}$$

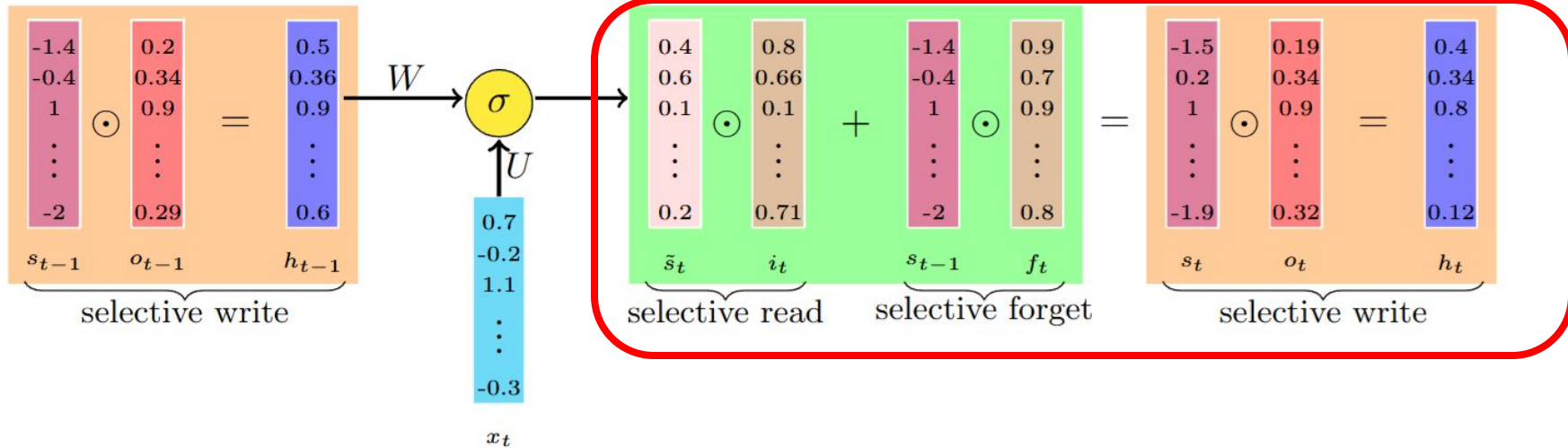
States:

$$\begin{aligned}
 \tilde{s}_t &= \sigma(W_{\tilde{s}} h_{t-1} + U_{\tilde{s}} x_t + b_{\tilde{s}}) \\
 s_t &= f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \\
 h_t &= o_t \odot \sigma(s_t)
 \end{aligned}$$

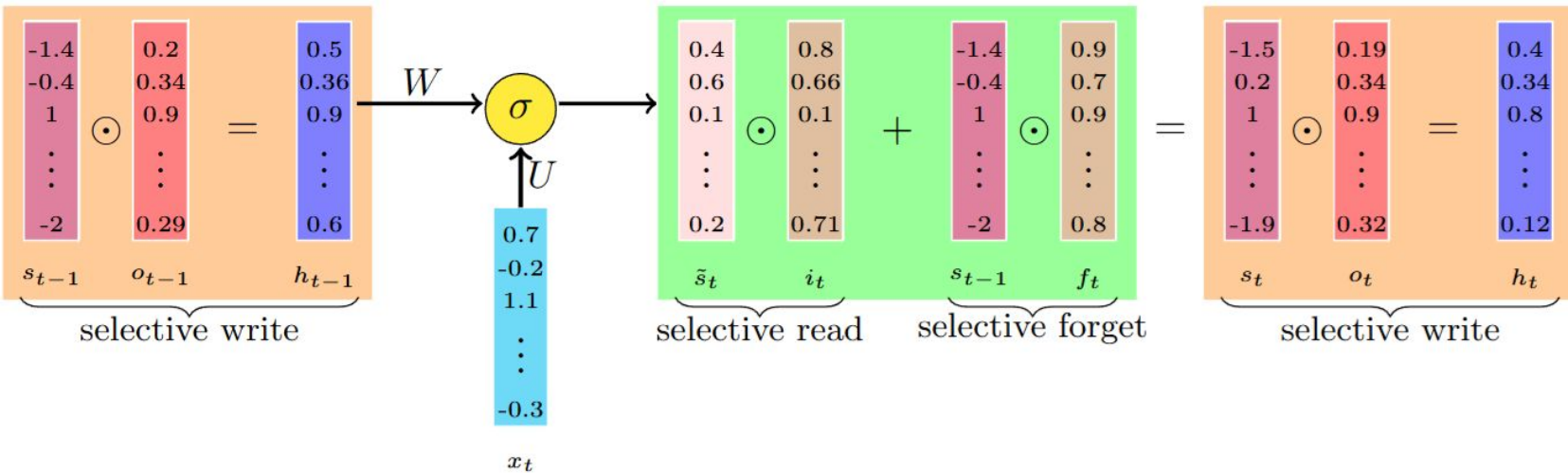
These are the set of equations for LSTM.



Identify what computations are happening at timestep t ?

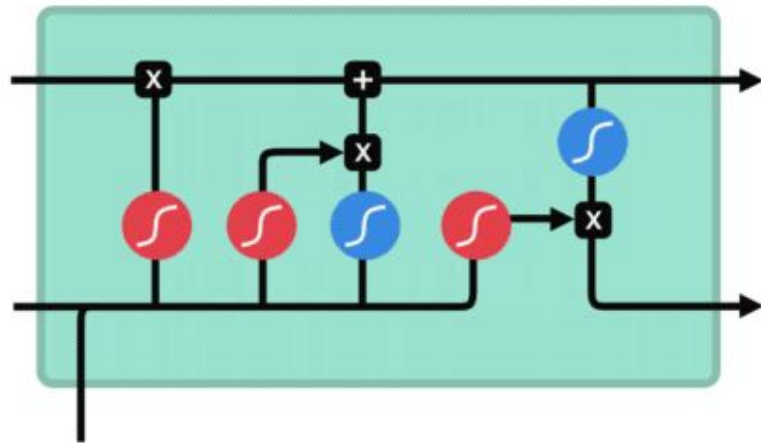
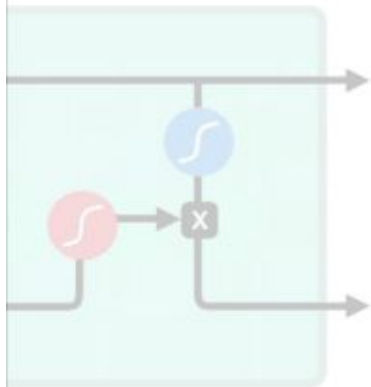


Identify what computations are happening at timestep t ?

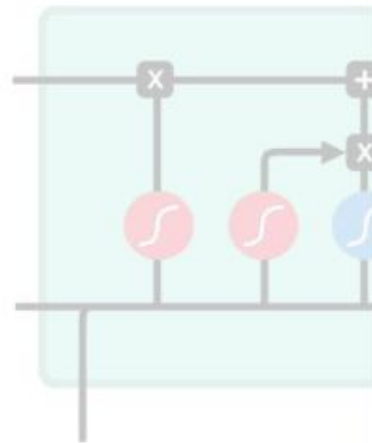


s_t : Cell state (or c_t)

h_t : Hidden state



0.2
0.5
1



Formal LSTM architecture

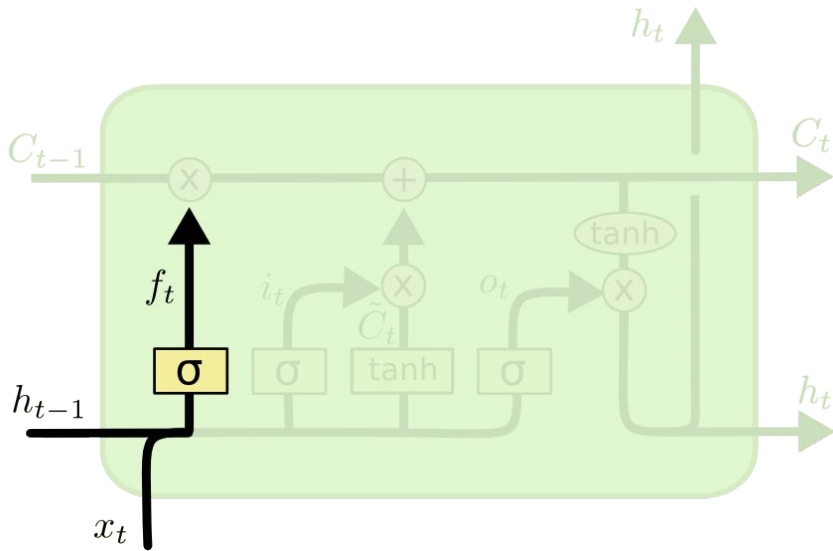
Forget gate

First, we have the forget gate. This gate decides what information should be thrown away or kept. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

Forget gate

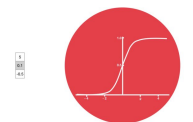
Decide what information we need to throw away or forget.

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

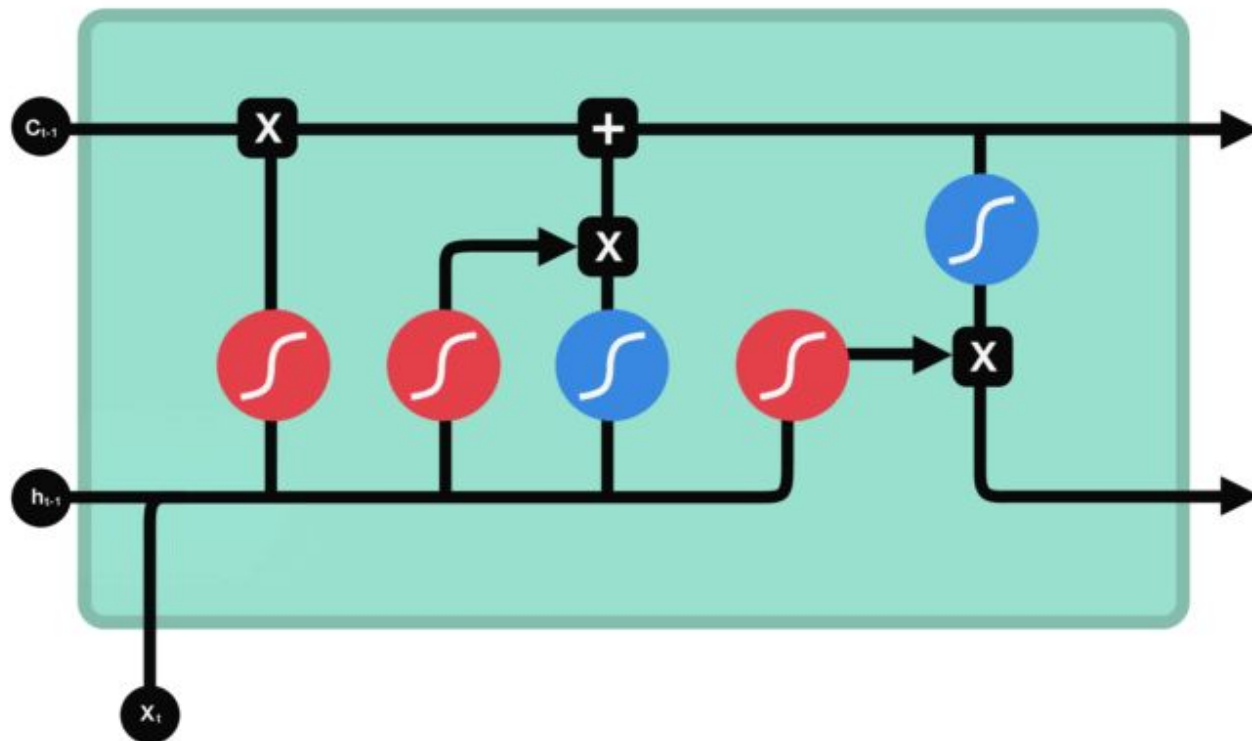


c_{t-1} previous cell state

f_t forget gate output



sigmoid



Input Gate

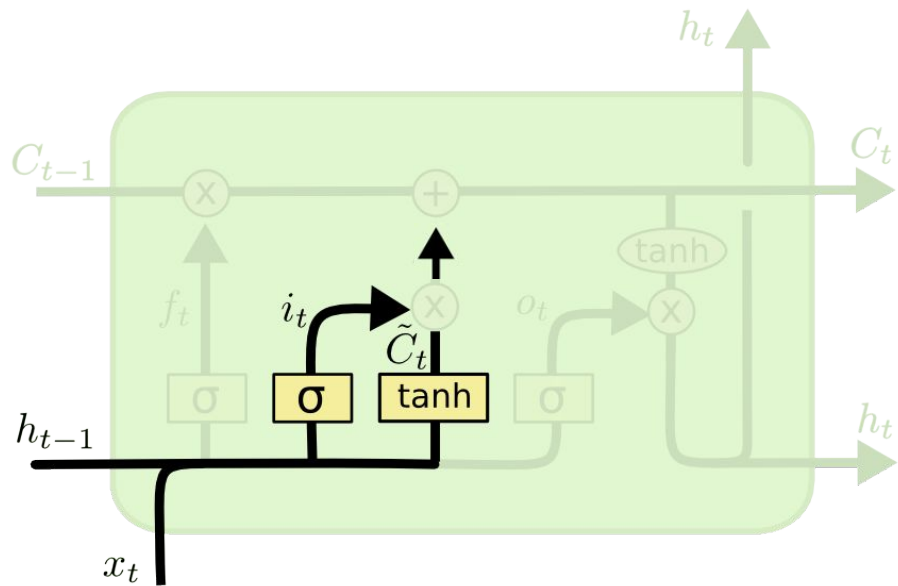
To update the cell state, we have the input gate. First, we pass the previous hidden state and current input into a sigmoid function. That decides which values will be updated by transforming the values to be between 0 and 1. 0 means not important, and 1 means important. You also pass the hidden state and current input into the tanh function to squish values between -1 and 1 to help regulate the network. Then you multiply the tanh output with the sigmoid output. The sigmoid output will decide which information is important to keep from the tanh output.

Input gate

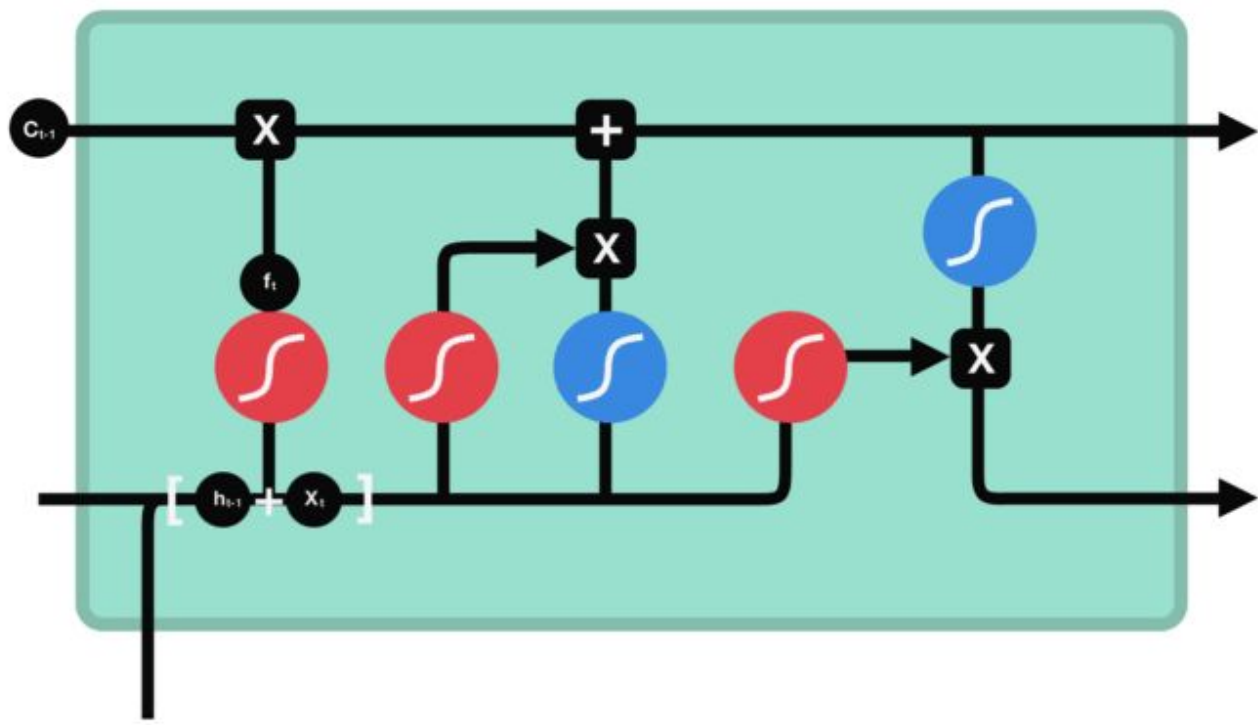
Decide what information we need to include in the cell state.

$$\begin{aligned} i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i) \\ \tilde{s}_t \text{ or } \tilde{C}_t &= \text{tanh}(W h_{t-1} + U x_t + b) \end{aligned}$$

tanh or σ both are equally valid.



- c_{t-1} previous cell state
- f_t forget gate output
- i_t input gate output
- $\tilde{c}_t$ candidate



Cell State

Now we should have enough information to calculate the cell state. First, the cell state gets pointwise multiplied by the forget vector. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0. Then we take the output from the input gate and do a pointwise addition which updates the cell state to new values that the neural network finds relevant. That gives us our new cell state.

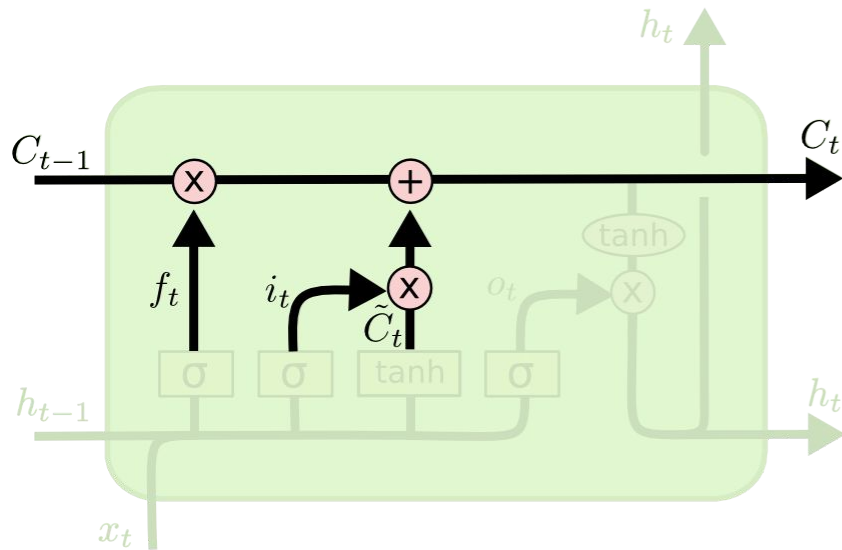
Update cell state

Update the old cell state

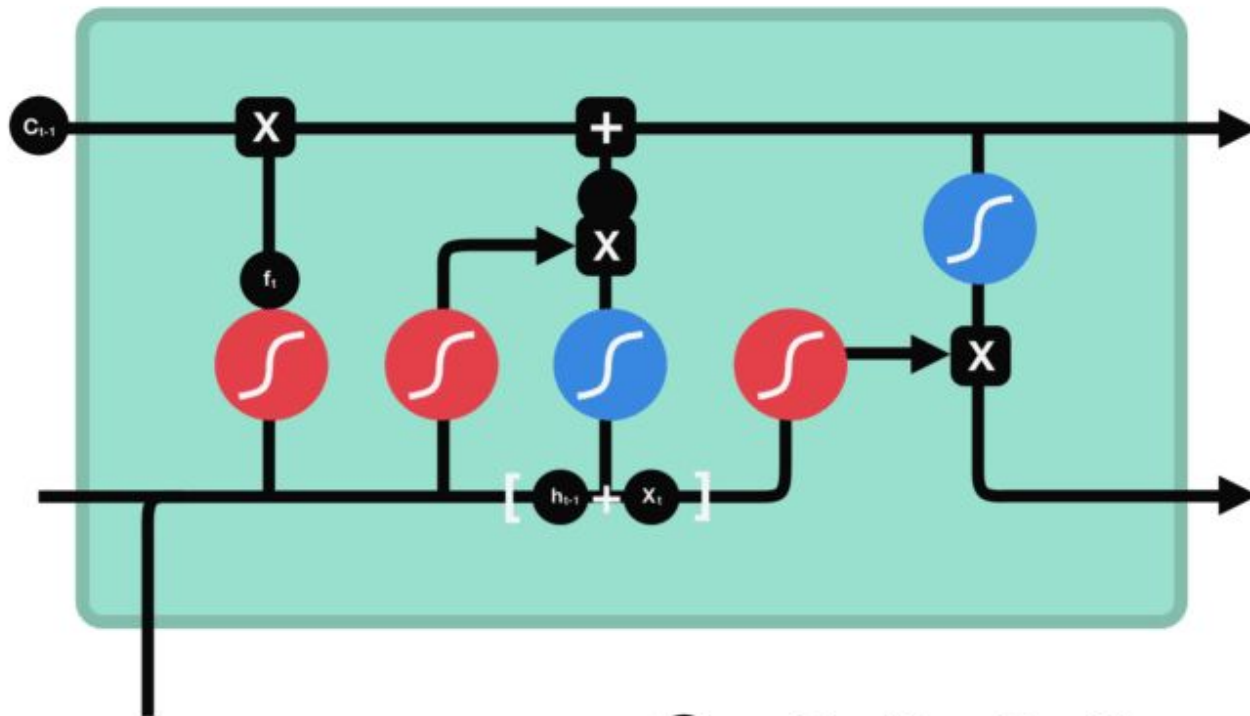
C_{t-1} or s_{t-1} into the new cell state

C_t or s_t .

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$



- c_{t-1} previous cell state
- f_t forget gate output
- i_t input gate output
- $\tilde{c}_t$ candidate
- c_t new cell state



$$c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$$

Output Gate

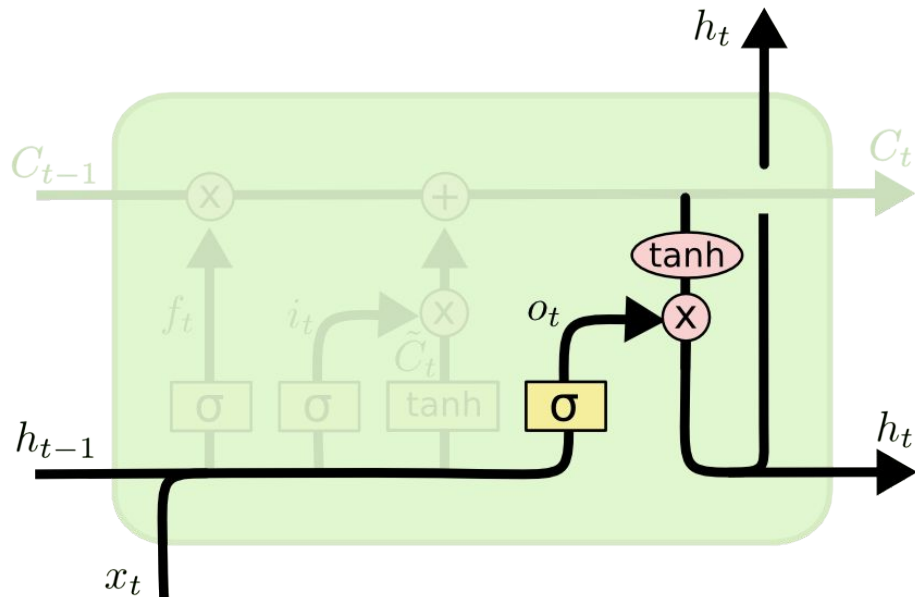
Last we have the output gate. The output gate decides what the next hidden state should be. Remember that the hidden state contains information on previous inputs. The hidden state is also used for predictions. First, we pass the previous hidden state and the current input into a sigmoid function.

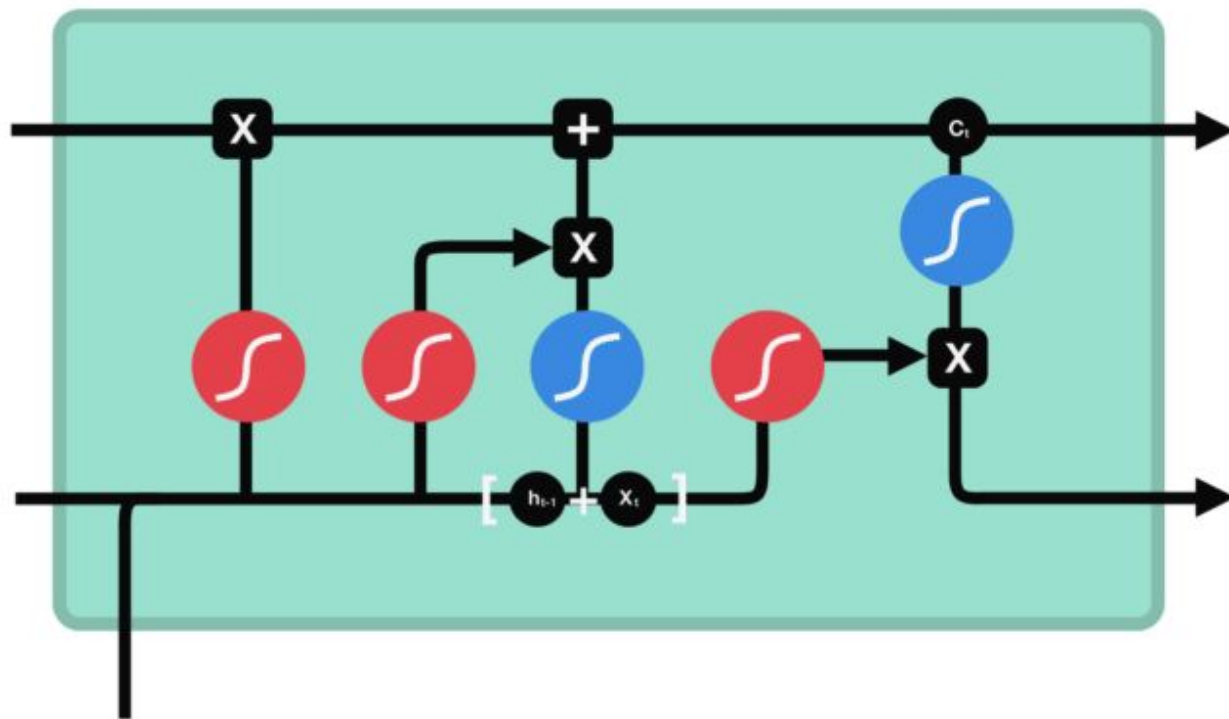
Then we pass the newly modified cell state to the tanh function. We multiply the tanh output with the sigmoid output to decide what information the hidden state should carry. The output is the hidden state. The new cell state and the new hidden is then carried over to the next time step.

Output gate

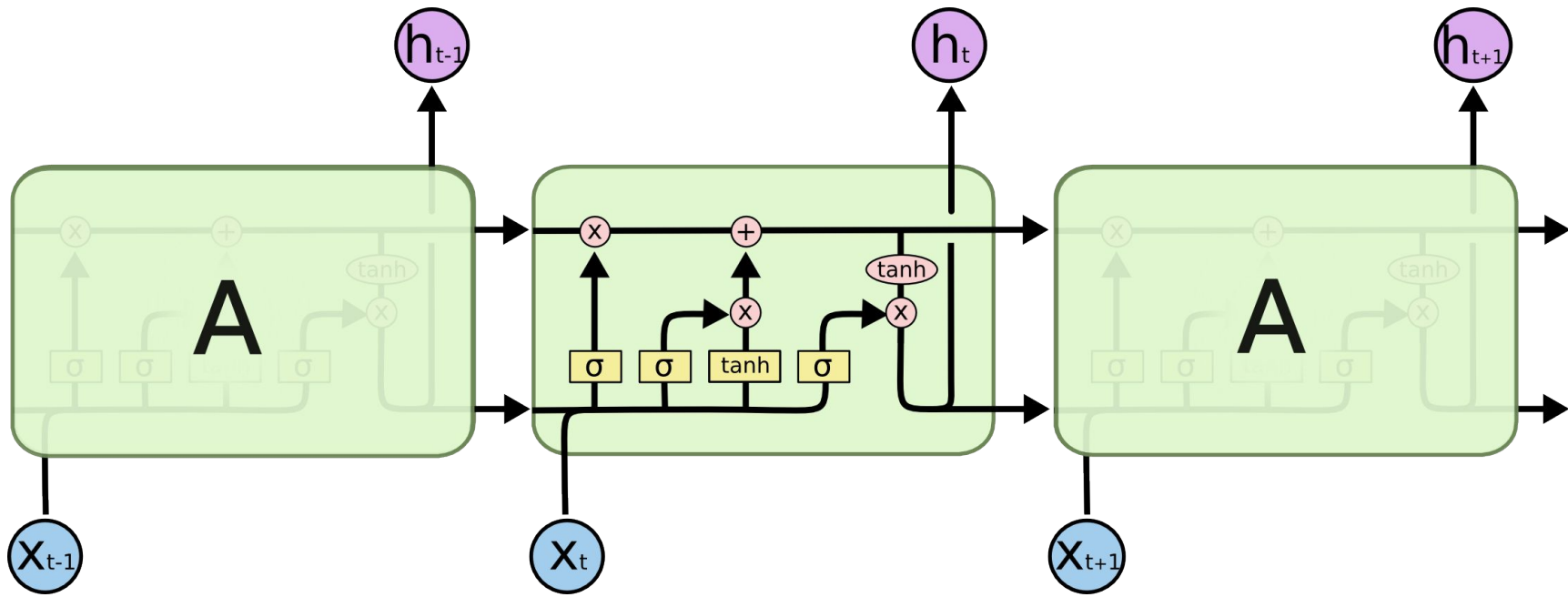
What to output based on the cell state C_t or s_t ?

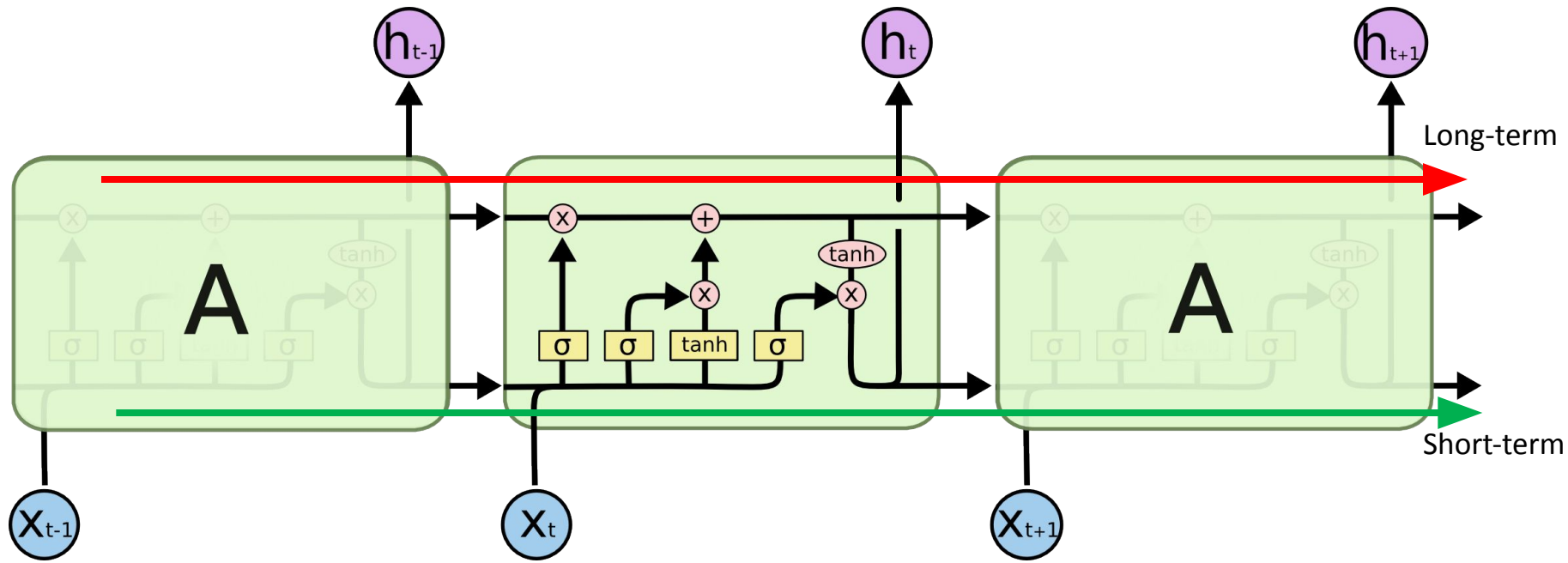
$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$





- c_{t-1} previous cell state
- f_t forget gate output
- i_t input gate output
- $\tilde{c}_t$ candidate
- c_t new cell state
- o_t output gate output
- h_t hidden state

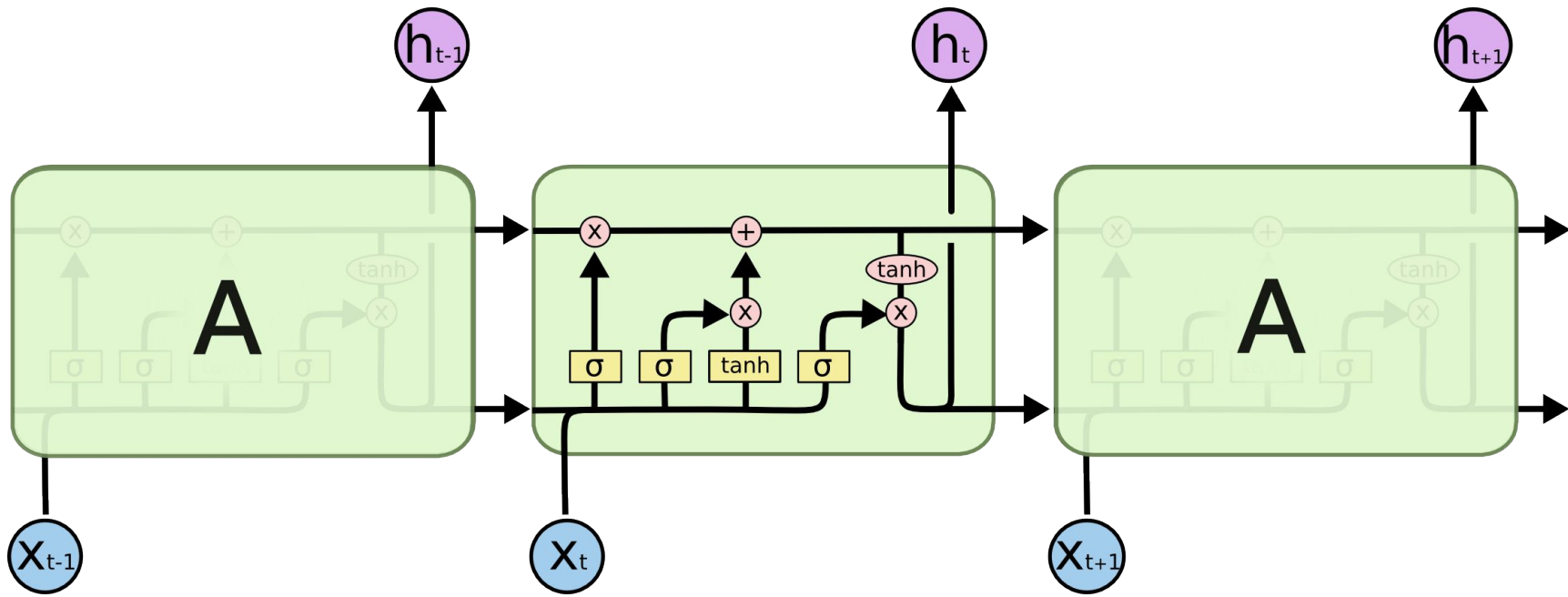




s_t : Cell state (or c_t) $\rightarrow$ This maintains Long-term context.

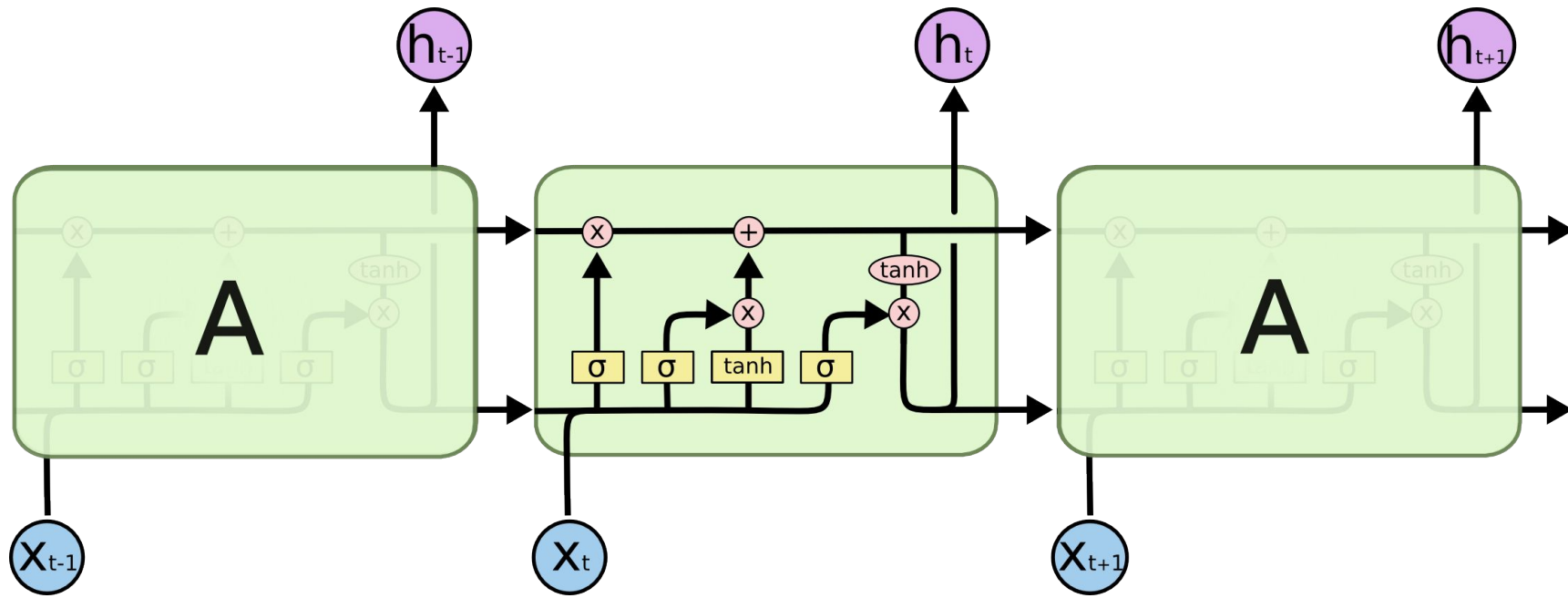
h_t : Hidden state $\rightarrow$ This maintains Short-term context.

Hence the name Long Short Term Memory



Note

s_t : Cell state (or c_t) and h_t : Hidden state will always have the same dimensions.



LSTM Cell

https://www.reddit.com/r/learnmachinelearning/comments/smindi/lstm_visualized/

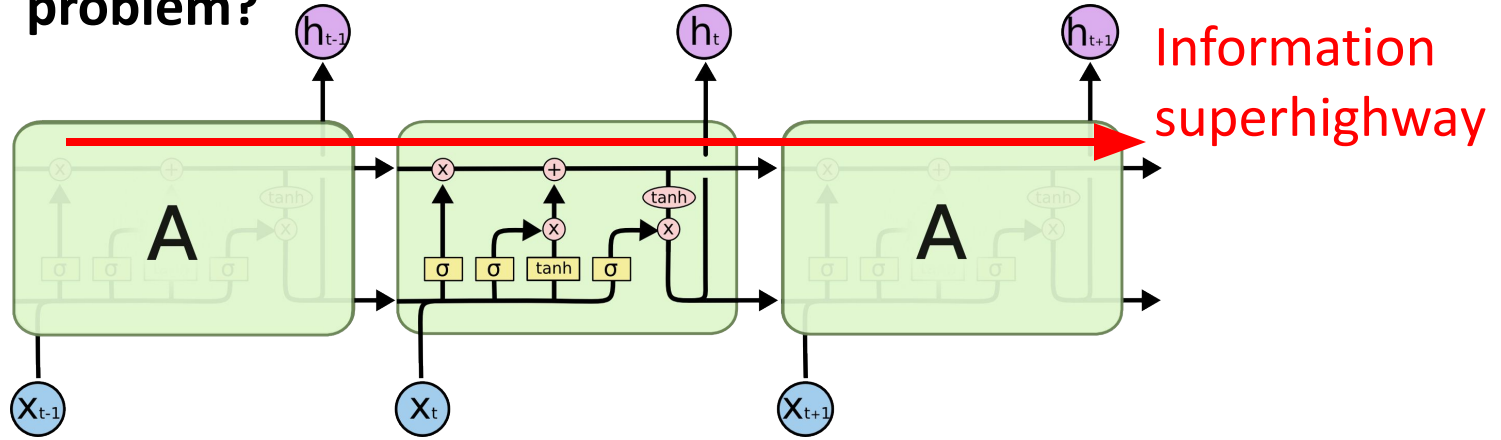
```
def LSTMCELL(prev_ct, prev_ht, input):
    combine = prev_ht + input
    ft = forget_layer(combine)
    candidate = candidate_layer(combine)
    it = input_layer(combine)
    Ct = prev_ct * ft + candidate * it
    ot = output_layer(combine)
    ht = ot * tanh(Ct)
    return ht, Ct
```

```
ct = [0, 0, 0]
ht = [0, 0, 0]
```

```
for input in inputs:
    ct, ht = LSTMCELL(ct, ht, input)
```

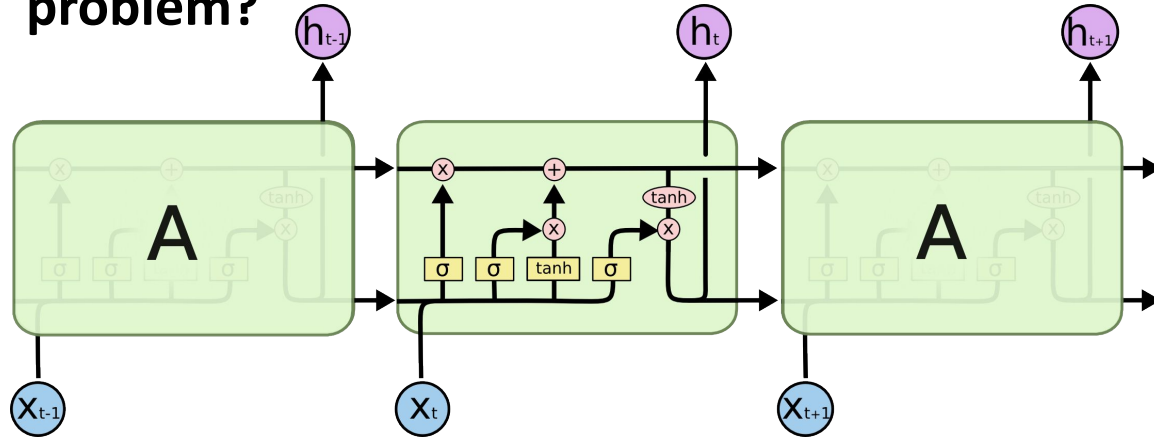
1. First, the previous hidden state and the current input get concatenated. We'll call it *combine*.
2. *Combine* get's fed into the forget layer. This layer removes non-relevant data.
4. A candidate layer is created using *combine*. The candidate holds possible values to add to the cell state.
3. *Combine* also get's fed into the input layer. This layer decides what data from the candidate should be added to the new cell state.
5. After computing the forget layer, candidate layer, and the input layer, the cell state is calculated using those vectors and the previous cell state.
6. The output is then computed.
7. Pointwise multiplying the output and the new cell state gives us the new hidden state.

How LSTMs solve vanishing gradient problem?



- C_t or s_t does not pass through any non-linear operations directly on the information superhighway.
- The **addition** operation prevents the gradient information from vanishing in the backward pass.
- Hence long-term memory is maintained for longer sequences.

How LSTMs solve vanishing gradient problem?



- In the **forward pass**, the gates enable only relevant information to flow.
- In the **backward pass**, the same gates will control the flow of gradient.
- If state at $t - 1$ does not contribute much to state at t , then during backprop gradient will not flow to C_{t-1} .

Gated Recurrent Unit



Recap – Problems in RNN

- - Encoding bottleneck (hidden state - h_t)
 - Slow to train, no parallelization
 - Long-term dependency issue.
-

Recap – Improvements made by LSTM

- Maintains separate long-term and short-term memory.
- Selective read, write and forget using gates.
- Solves long-term dependency issue.
- Solves vanishing gradient problem.
- Stable training, performs well on longer sequences.



Drawbacks of LSTM

- Large number of parameters due to multiple gates, each having their weights and biases.
 - High computational cost.
 - Complex architecture.
 - Slow to train.
 - Still struggles with very long sequences.
-

Can we have a simpler alternative of LSTM?

Can we have a simpler alternative of LSTM?

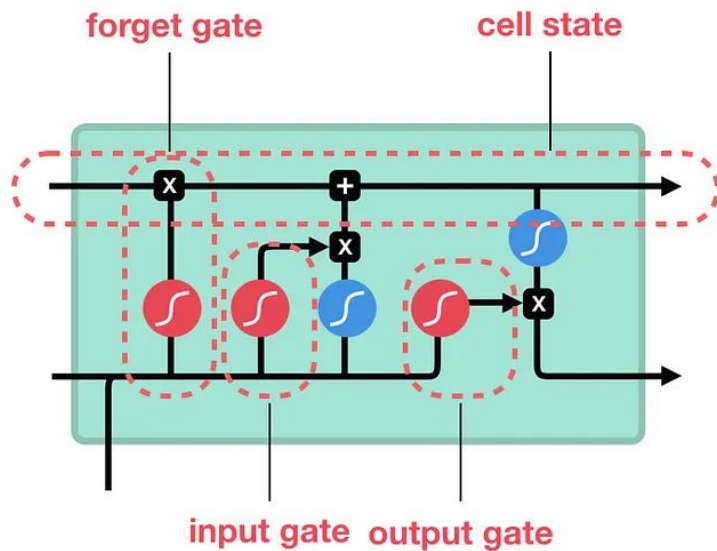
YES

In 2014, GRU was proposed as a lighter alternative of LSTM with comparable performance.

GRU

So now we know how an LSTM work, let's briefly look at the GRU. The GRU is the newer generation of Recurrent Neural networks and is pretty similar to an LSTM. GRU's got rid of the cell state and used the hidden state to transfer information. It also only has two gates, a reset gate and update gate.

LSTM



sigmoid



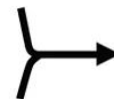
tanh



pointwise
multiplication

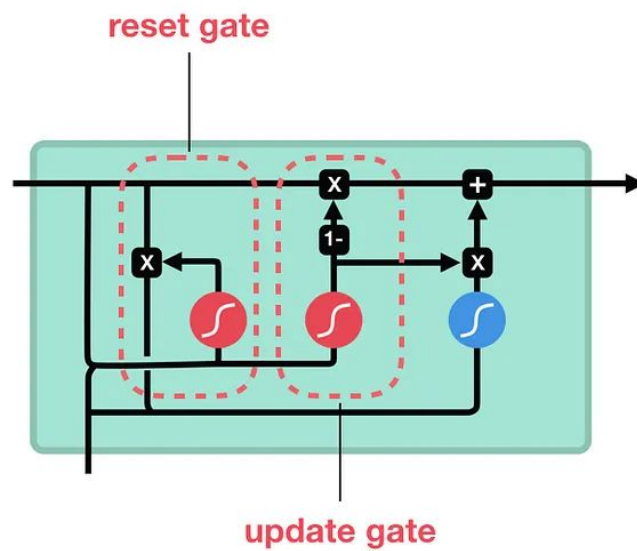


pointwise
addition



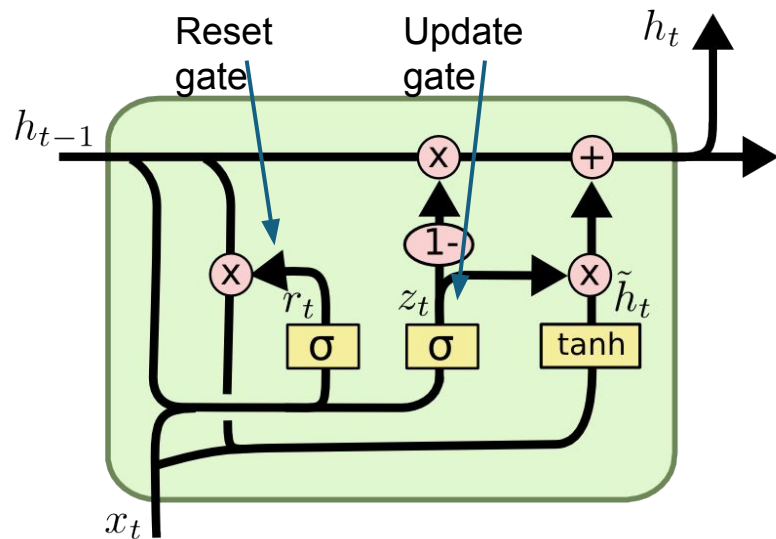
vector
concatenation

GRU

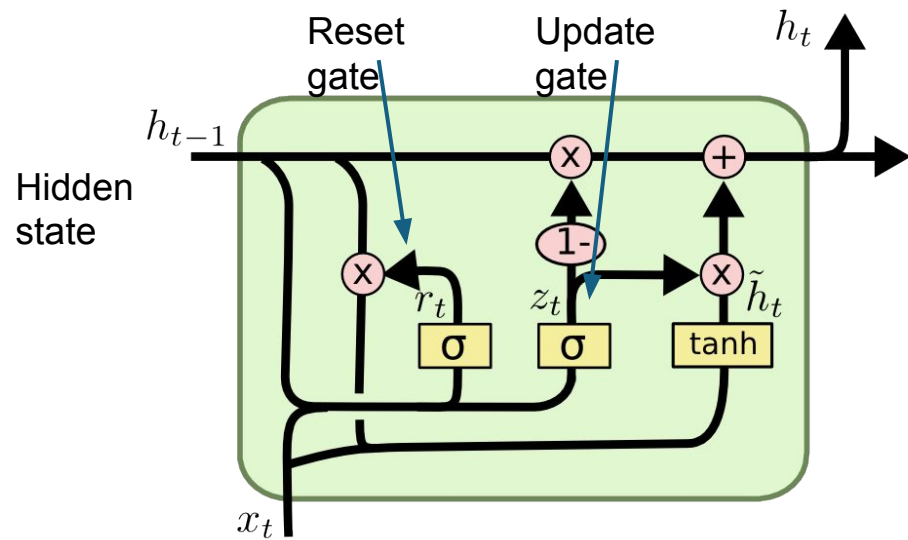
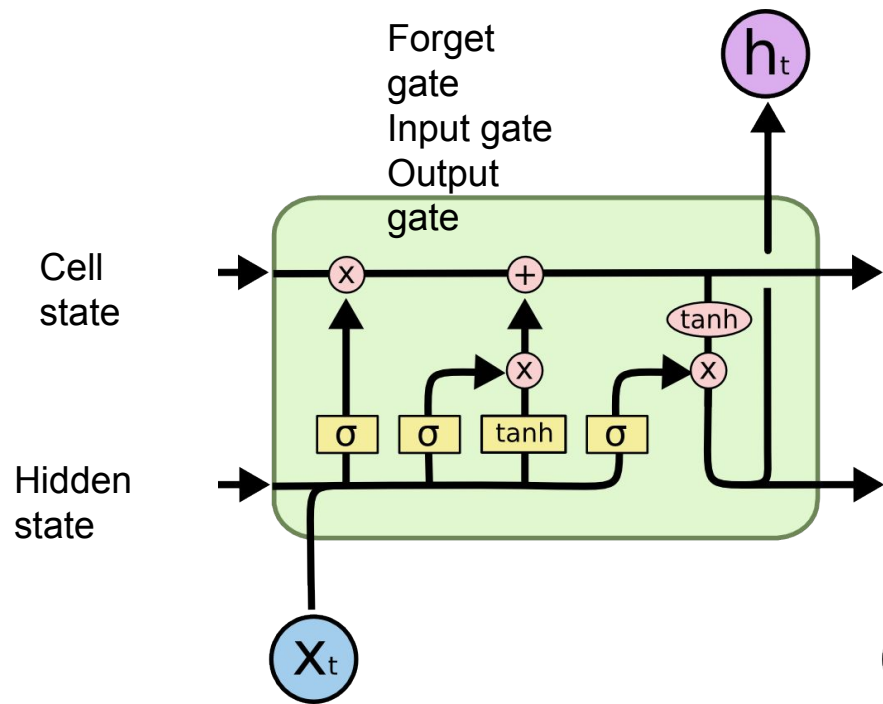


GRU

- It has only one state.
- And only two gates:
 - Reset gate
 - Update gate



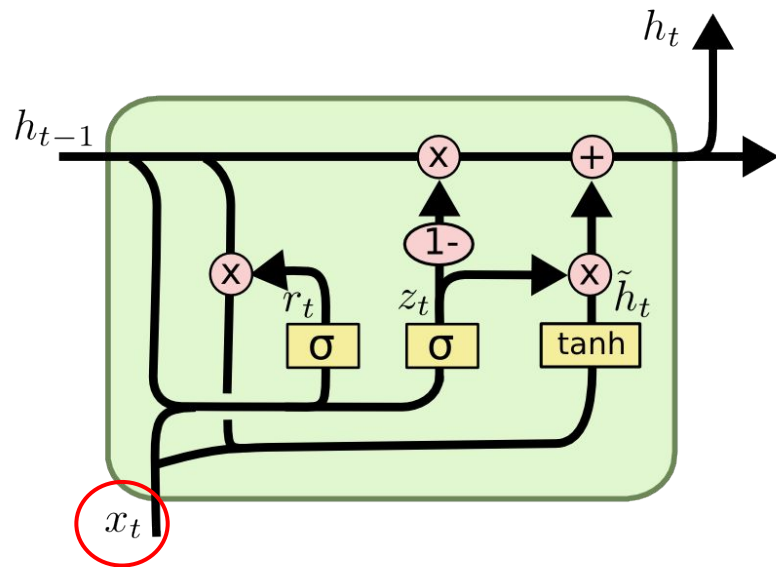
LSTM vs GRU: A high-level contrast



Unpacking the architecture.....

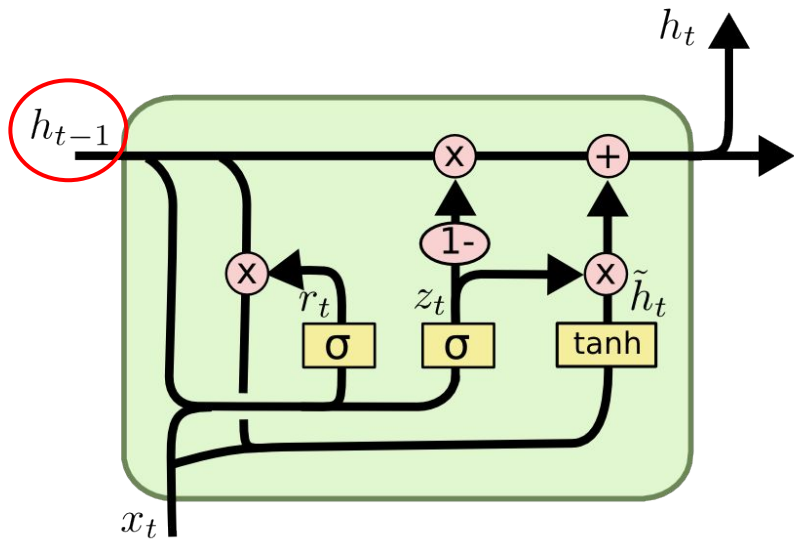
Components of the GRU Cell

- x_t : Input at current timestep



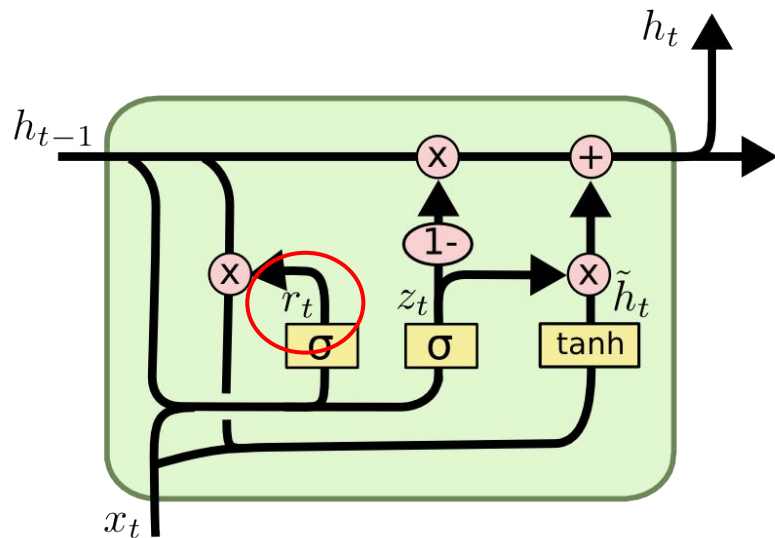
Components of the GRU Cell

- x_t : Input at current timestep
- h_{t-1} : Hidden state from previous time step or *previous memory*



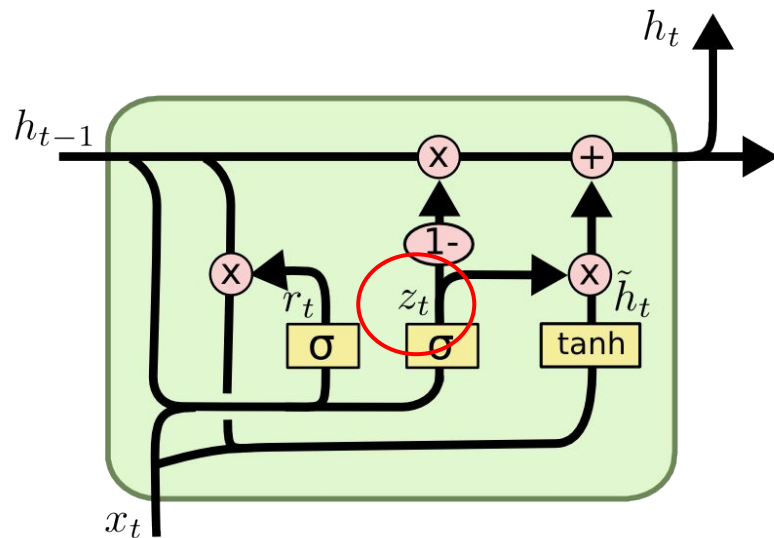
Components of the GRU Cell

- x_t : Input at current timestep
- h_{t-1} : Hidden state from previous time step or *previous memory*
- r_t : Reset gate, how much of the previous memory should be forgotten.



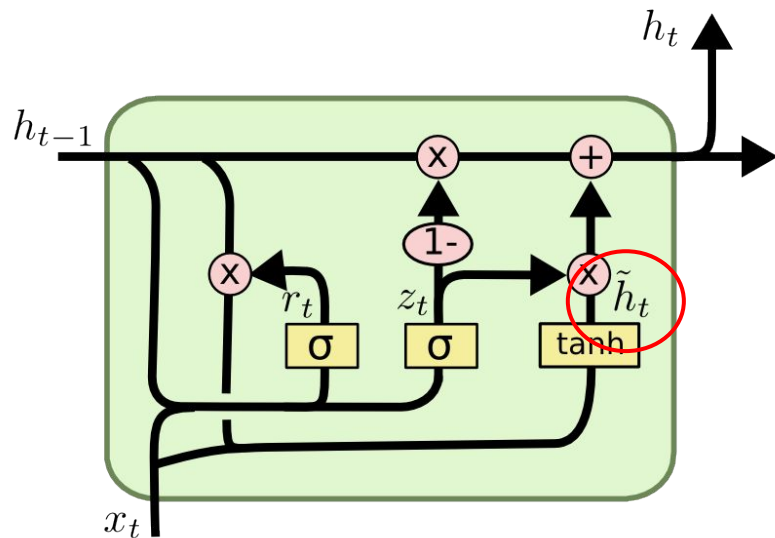
Components of the GRU Cell

- x_t : Input at current timestep
- h_{t-1} : Hidden state from previous time step or *previous memory*
- r_t : Reset gate, how much of the previous memory should be forgotten.
- z_t : Update gate – tells considering new information, how should old memory be updated.



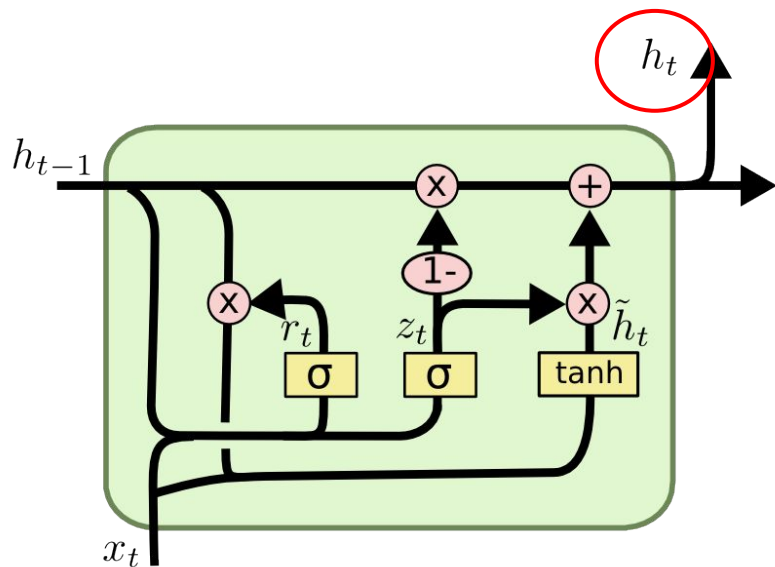
Components of the GRU Cell

- x_t : Input at current timestep
- h_{t-1} : Hidden state from previous time step or *previous memory*
- r_t : Reset gate, how much of the previous memory should be forgotten.
- z_t : Update gate – tells considering new information, how should old memory be updated.
- $\tilde{h}_t$: Temporary hidden state after considering new information.



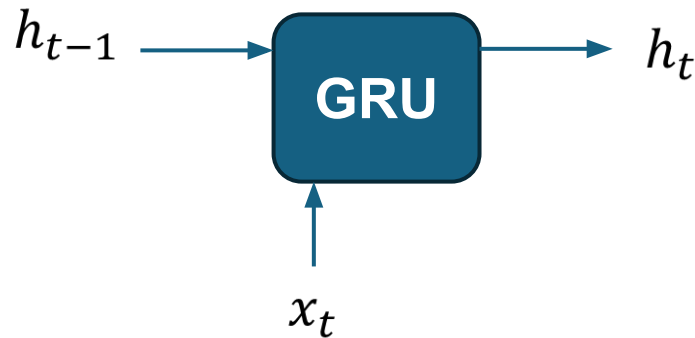
Components of the GRU Cell

- x_t : Input at current timestep
- h_{t-1} : Hidden state from previous time step or *previous memory*
- r_t : Reset gate, how much of the previous memory should be forgotten.
- z_t : Update gate – tells considering new information, how should old memory be updated.
- $\tilde{h}_t$: Temporary hidden state after considering new information.
- h_t : Output of current timestep.



Goal of GRU

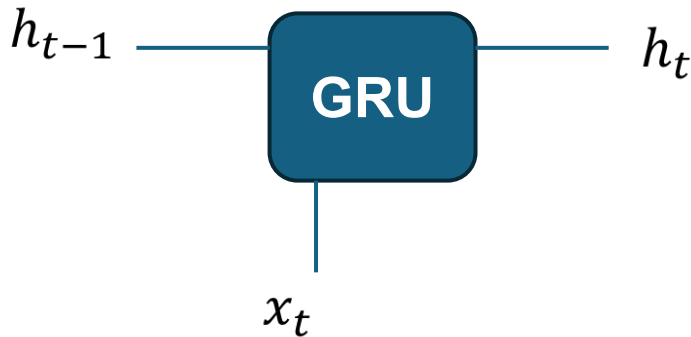
- GRU takes two inputs:
 - h_{t-1} - previous memory *or* context learnt
 - x_t - new information received



Goal of GRU

- GRU takes two inputs:
 - h_{t-1} - previous memory or context learnt
 - x_t - new information received
- Considering
 - this previous memory and
 - current information,

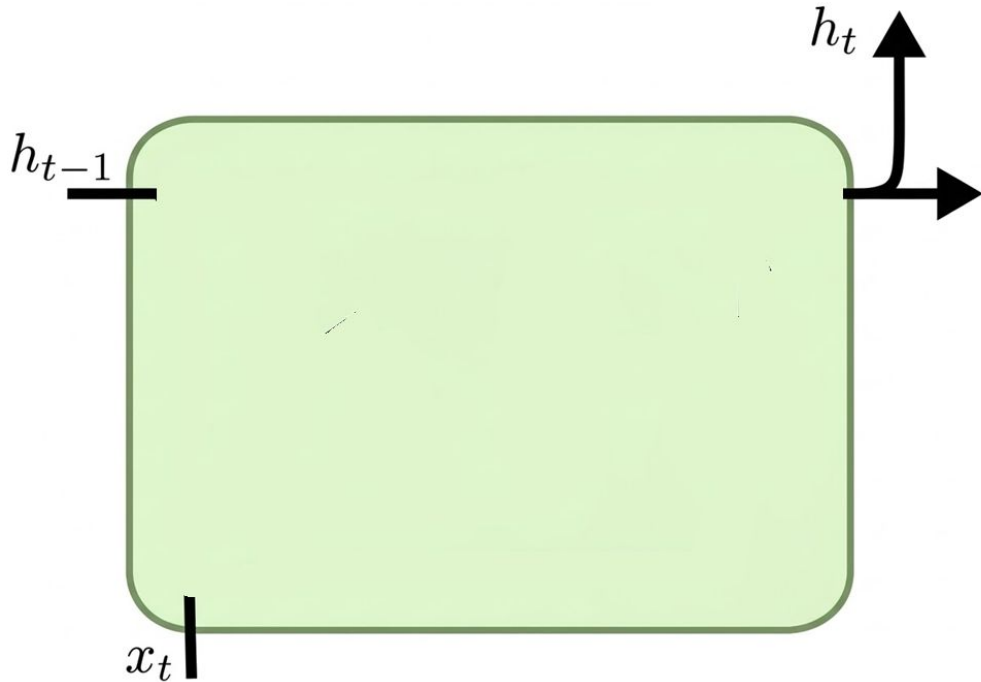
GRU will produce updated hidden state (updated memory) to be carried forward.



Step-by-step computation

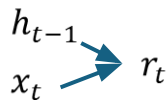
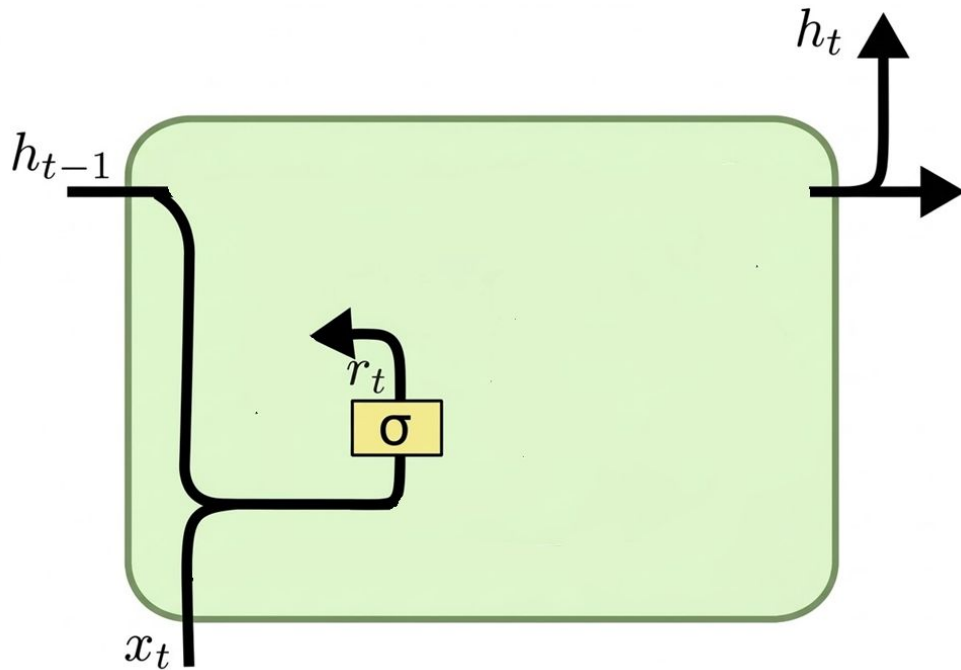
- Take inputs:

- h_{t-1} : Previous memory
- x_t : Current input

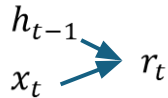
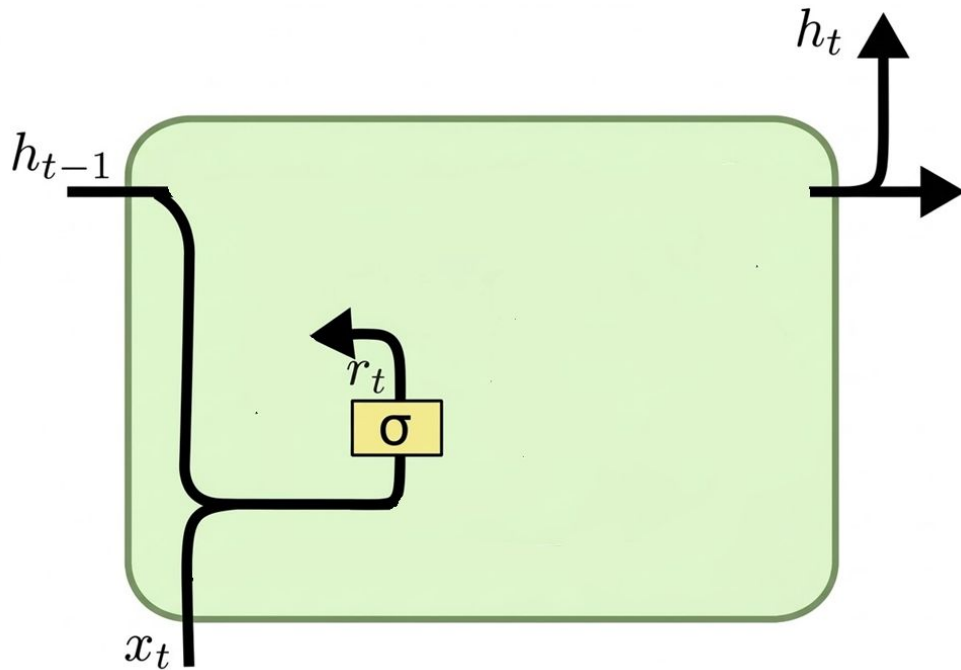


h_{t-1}
 x_t

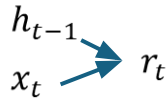
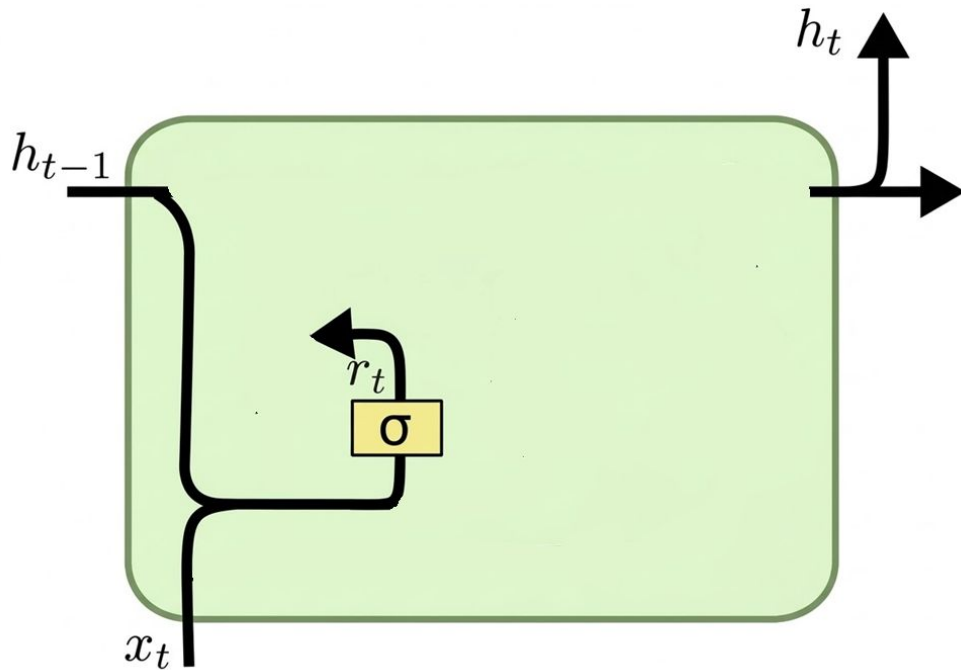
- We want to **forget selectively** from previous memory h_{t-1} .
- **But how to know what to forget?**



- We want to **forget selectively** from previous memory h_{t-1} .
- Introduce sigmoid gate with learnable parameters.
- This sigmoid layer will give vector r_t .
- It will act as a reset gate.



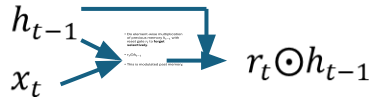
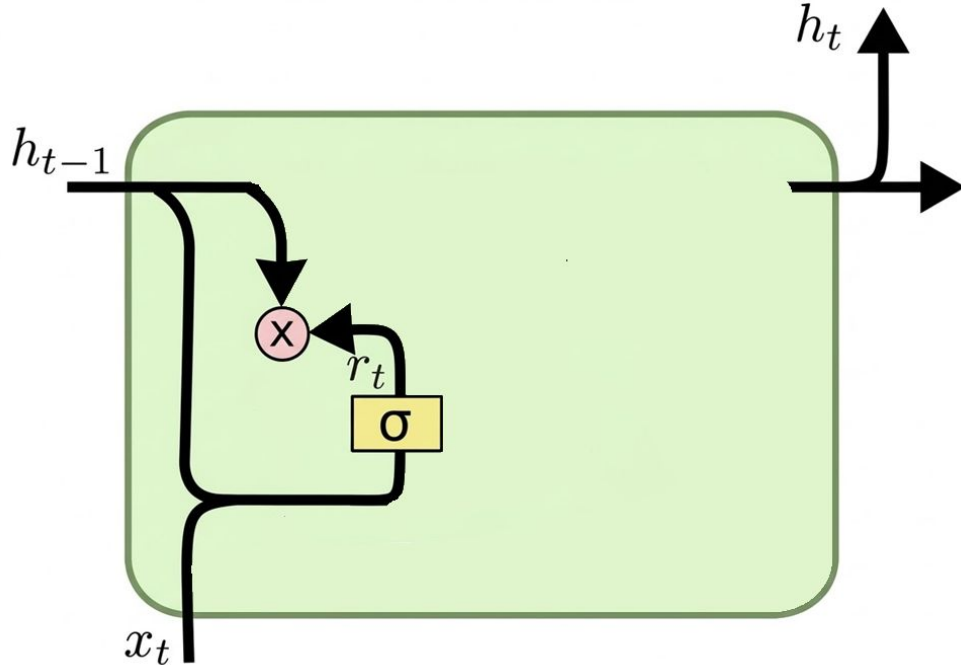
- We want to **forget selectively** from previous memory h_{t-1} .
- Introduce sigmoid gate with learnable parameters.
- This sigmoid layer will give vector r_t .
- It will act as a reset gate.
- $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
- W_r, U_r, b_r will be trained to forget selectively.



- Do element-wise multiplication of previous memory h_{t-1} with reset gate r_t to **forget selectively**.

- $r_t \odot h_{t-1}$

- This is modulated past memory.

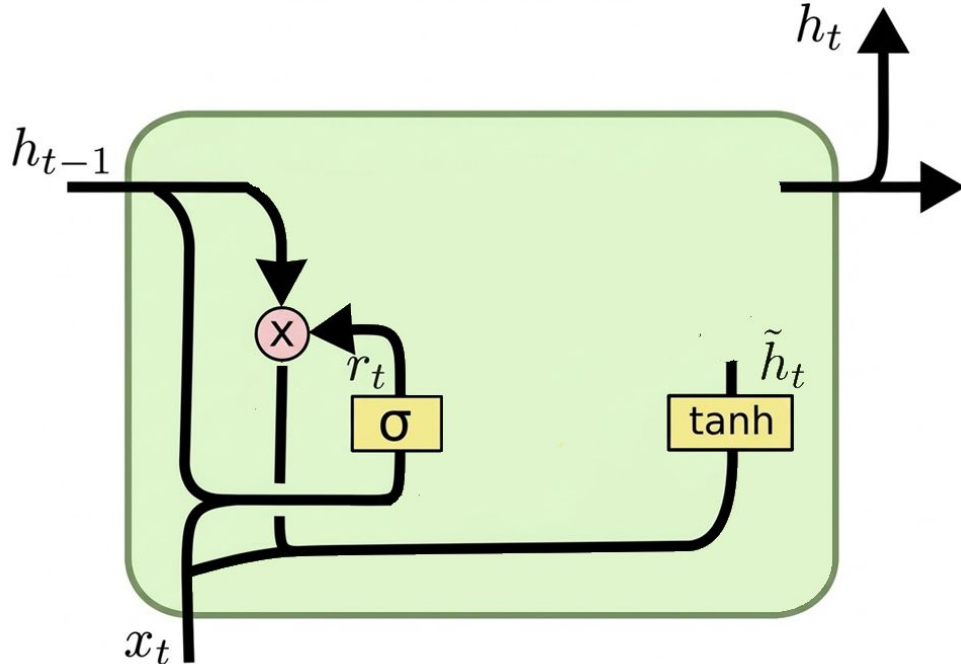


- Using past modulated memory and current input, calculate temporary hidden state.

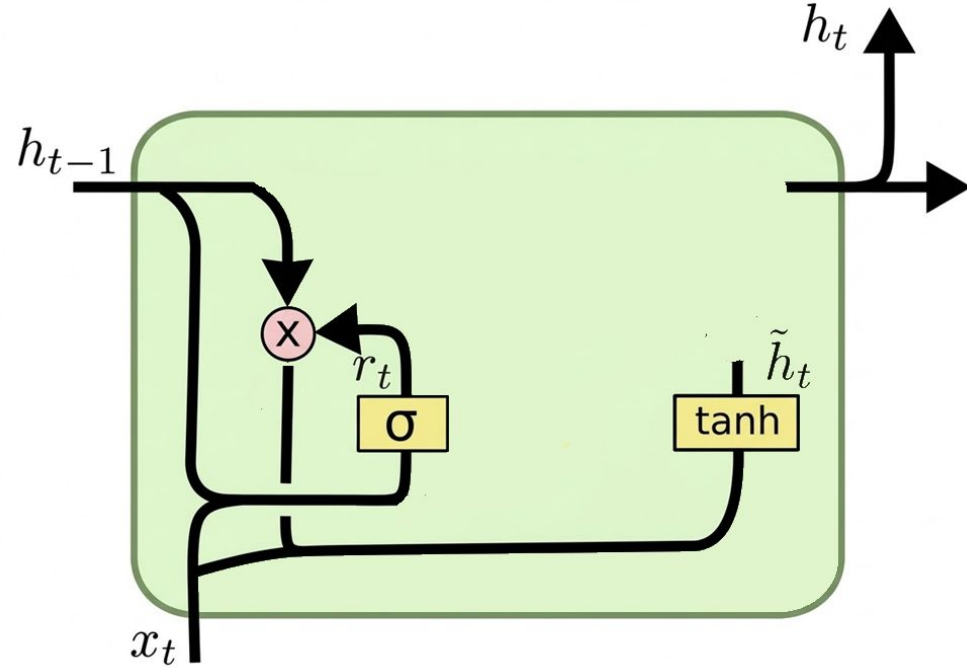
- $h_{t-1} \odot r_t$ - Modulated memory

- $\tilde{h}_t$ - Temporary hidden state.

- $\tilde{h}_t$ contains information from filtered past and current input.

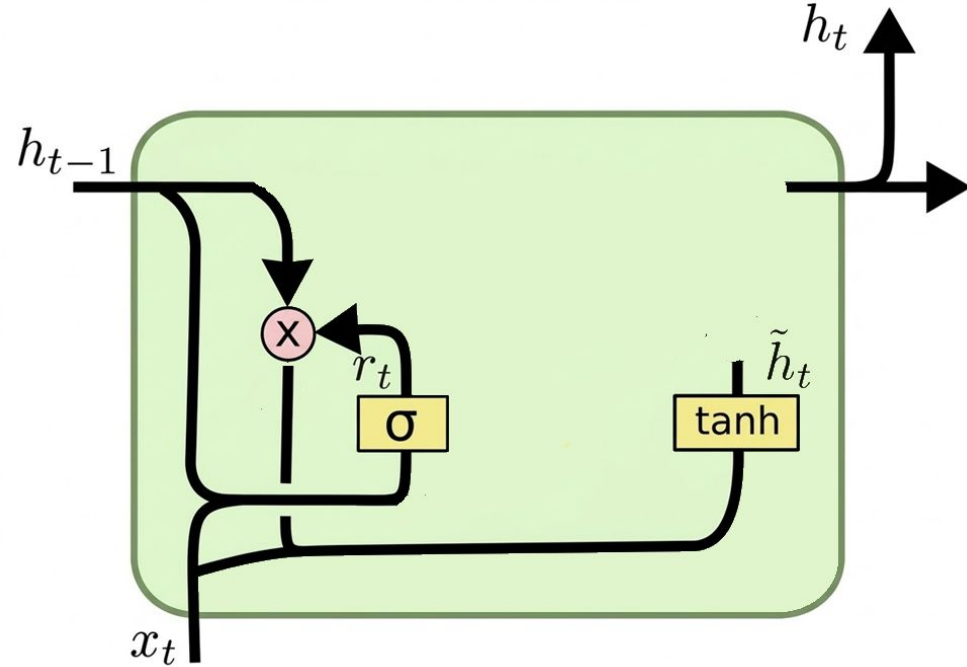


- If $\tilde{h}_t$ contains information from filtered past and current input, why don't we carry it forward as h_t (output hidden state)?

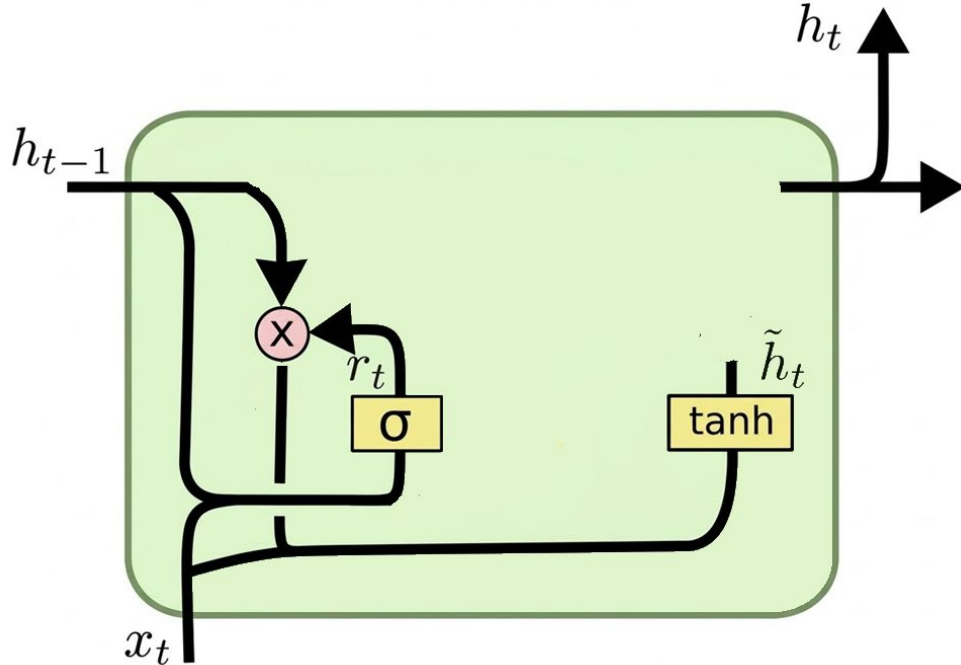


- If $\tilde{h}_t$ contains information from filtered past and current input, why don't we carry it forward as h_t (output hidden state)?

- **Because this is heavily influenced by current input, which may not all be relevant for long term context.**

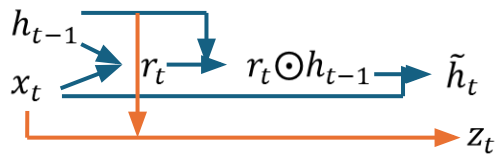
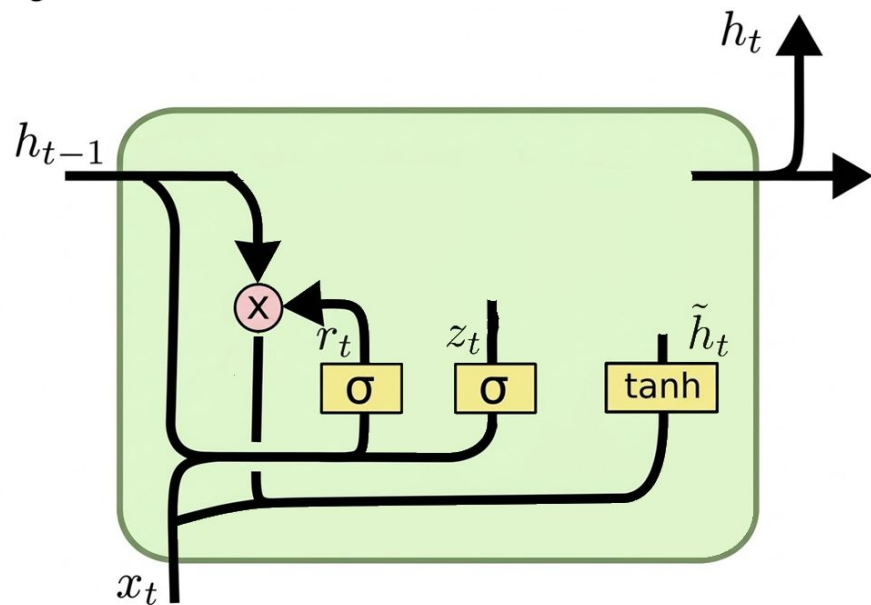


- $h_{t-1} \odot r_t$ - Modulated memory
- $\tilde{h}_t$ - Temporary hidden state.
- $\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
- W_h, U_h & b_h are learnable parameters.



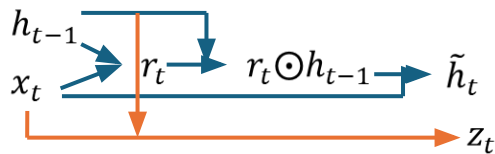
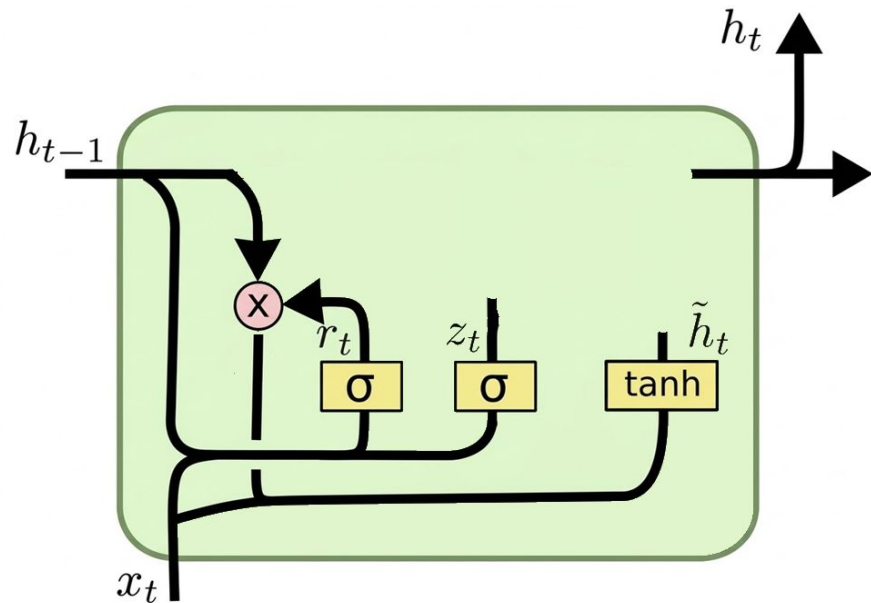
Now calculate Update gate z_t

- We have,
 - h_{t-1} - Previous memory
 - $\tilde{h}_t$ - Temporary hidden state
- Update gate controls how much of past memory matters now.
- z_t will learn:
 - What is important in h_{t-1} to carry forward?
 - What is important in $\tilde{h}_t$ to carry forward?



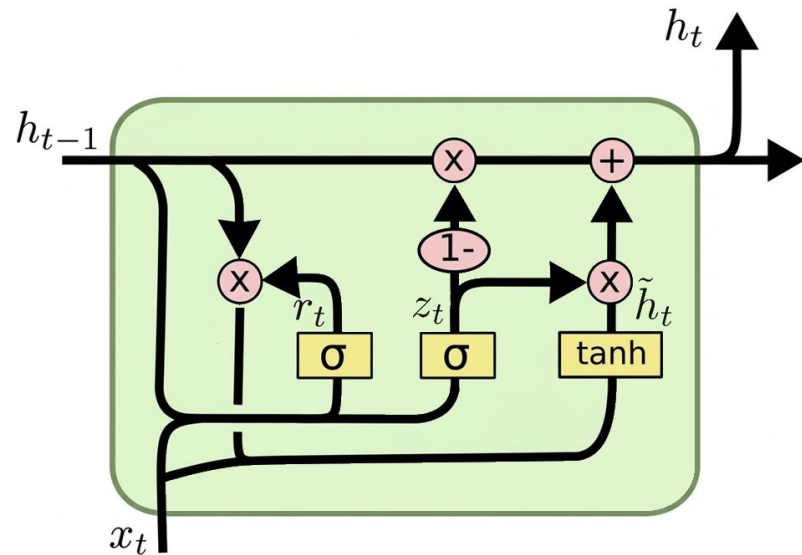
Now calculate Update gate z_t

- We have,
- h_{t-1} - Previous memory
- $\tilde{h}_t$ - Temporary hidden state
- $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
- z_t is essentially a vector which will decide the proportion of importance.

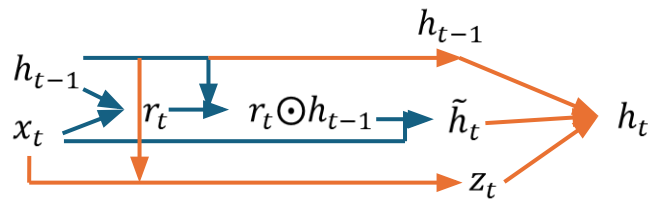


Use z_t , h_{t-1} & $\tilde{h}_t$ to find h_t

- We have,
- h_{t-1} - Previous memory
- $\tilde{h}_t$ - Temporary hidden state
- z_t - Update gate



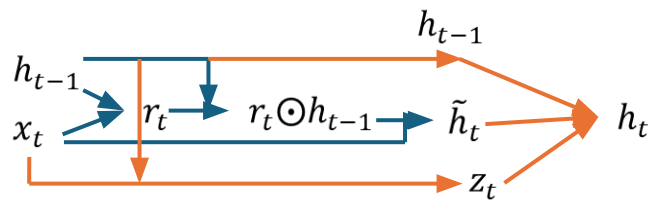
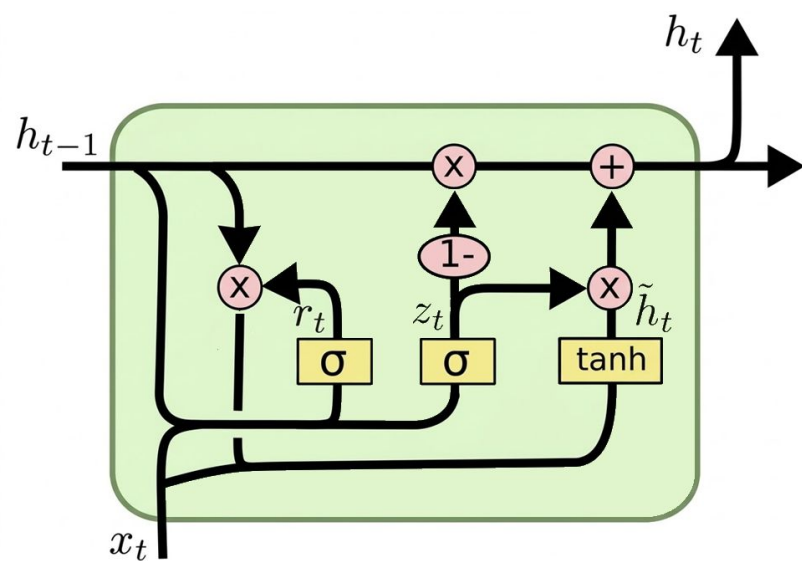
- Combine these to find h_t .



Use z_t , h_{t-1} & $\tilde{h}_t$ to find h_t

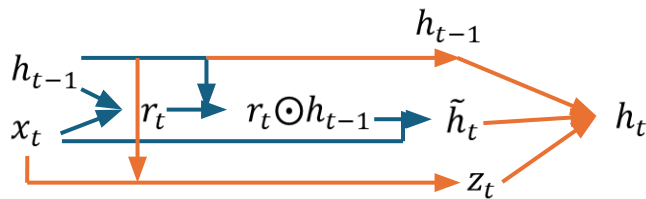
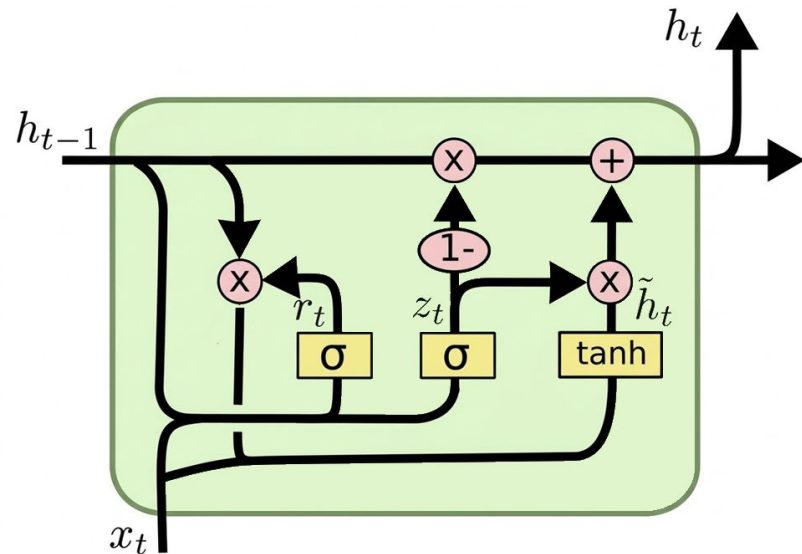
- We have,
- h_{t-1} - Previous memory
- $\tilde{h}_t$ - Temporary hidden state
- z_t - Update gate

$$h_t = (1 - z_t) \odot h_{t-1} \oplus z_t \odot \tilde{h}_t$$



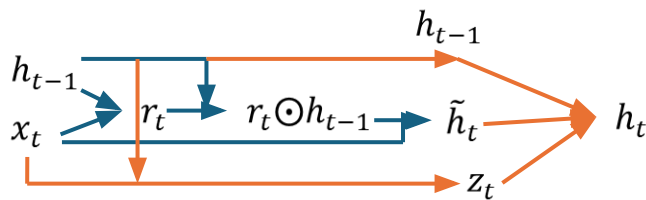
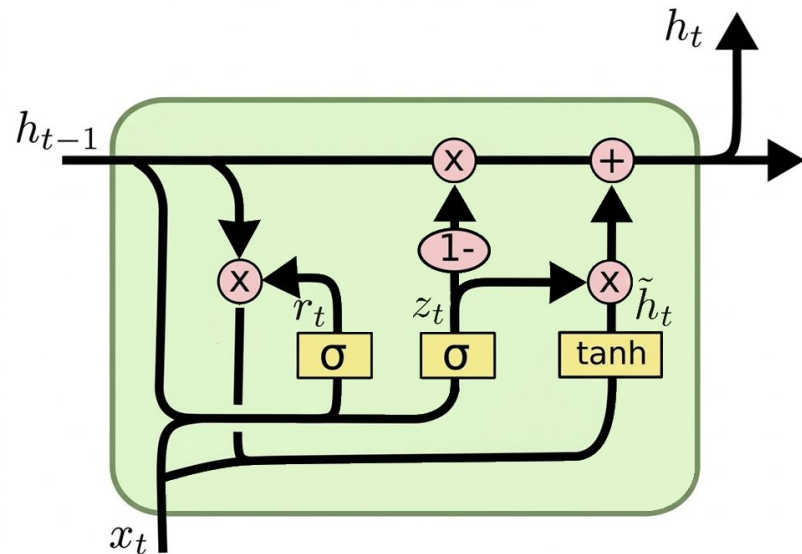
Use z_t , h_{t-1} & $\tilde{h}_t$ to find h_t

- $h_t = (1 - z_t) \odot h_{t-1} \oplus z_t \odot \tilde{h}_t$
- If $z_t \approx 0 \Rightarrow$ Only past memory is relevant:
- $h_t \approx h_{t-1}$



Use z_t , h_{t-1} & $\tilde{h}_t$ to find h_t

- $h_t = (1 - z_t) \odot h_{t-1} \oplus z_t \odot \tilde{h}_t$
- If $z_t \approx 0 \Rightarrow$ Only past memory is relevant:
- $h_t \approx h_{t-1}$
- If $z_t \approx 1 \Rightarrow$ Past memory is irrelevant:
- $h_t \approx \tilde{h}_t$



LSTM vs GRU

- GRUs have fewer parameters than LSTM.
- LSTM can be the default choice, but GRU can give comparable performance.
- If data has long-range dependency and training data is large, LSTM can be better.
- Whenever speed and computational cost is a concern, switch to GRUs.